

Talking Garments Plus: Large Language Model-Mediated Embedded Multi-Agent System for Unmanned Clothing-Store

SEIoT MSc Final Project 2025
University College London

ABSTRACT

The emergence of large language models has brought new development prospects to many applications. One of them is the multi-agent system development. This project presents a proof-of-concept unmanned clothing-store system that combines physically embodied agents with a cloud-hosted large language model to support natural, task-oriented dialogue. The design problem is enabling shoppers to receive guidance and item-specific information without staff, while maintaining legible interaction, reliable coordination, and low friction. The system comprises: (i) a mobile greeter and guide (Jarvis and Alice) built on Raspberry Pi 4 and a small mobile base, (ii) garment-embedded agents on Raspberry Pi 3 with microphones, speakers, touch and motion sensors, and small displays, and (iii) a central service for registration, health monitoring, conversation logging, and multi-agent handover. Speech interaction uses voice-activity detection and endpointing, streaming automatic speech recognition, model-driven dialogue via structured prompts and light-weight context scaffolds, and responsive text-to-speech with barge-in; components are modular and swappable.

We built and integrated the full stack (hardware, firmware, and Python services) and evaluated it through bench tests and a small, informal user study focused on user experience. The prototype demonstrates reliable agent discovery and handover, basic context retention across turns, and clear role separation between mobile and garment-embedded agents. Users reported that guidance clarity and physical legibility were strong, while two limitations remained

salient: end-to-end latency in speech interaction and occasional responses that felt insufficiently natural or slightly off-target. Navigation among multiple garment agents is currently heuristic and will be replaced with principled path planning over a store map.

The results indicate that an LLM-mediated, role-specialised, centrally orchestrated architecture is a viable blueprint for unmanned retail. The system is immediately extensible to local (on-device) language models, human-aware navigation, multi-modal item discovery (vision/RFID/BLE), and broader accessibility features, providing a practical pathway from prototype to deployable, scalable retail experiences.

Author Keywords

Large Language Models (LLMs) Application; Multi-agent System (MAS); Intelligent Agent Design; Internet of Things (IoTs); Embedded Sensor System; Human-Agent Interaction Design

Video and Github Page

You can find the project videos, instructions and source code at this Github Page: <https://github.com/dzy0803/TalkingGarments-Plus>

1 INTRODUCTION

The field of multi-agent systems emerged from distributed artificial intelligence, where agents with bounded capabilities coordinate to solve problems no single agent can handle alone. Early systems relied on symbolic rules, domain ontologies, and hand-engineered communication protocols to support cooperation, negotiation, and task allocation in constrained settings such as manufacturing, logistics, and simulation. The advent of large language models has shifted this paradigm. Because LLMs can parse instructions, reason over open-ended inputs, and produce coherent plans and dialogue, they offer a general substrate for agency that is adaptable rather than pre-scripted. Through prompt engineering—careful role framing, goal specification, constraints, and step

MSC SEIoT FINAL PROJECT REPORT

Project report submitted in part fulfilment of the requirements for the degree of Master of Science System Engineering for the Internet of Things, University College London, 2025.

NOTE BY THE UNIVERSITY

This project report is submitted as an examination paper. No responsibility can be held by University College London for the accuracy or completeness of the material therein.

structuring—LLMs can be steered toward reliable behaviour for a target task. Through context engineering—curating what the model “sees” at inference time via retrieval, tool outputs, memory, and recent interaction histories—LLMs can be grounded in local knowledge, current state, and environment feedback. Together, these two levers turn a general model into a task-conditioned agent that can collaborate with other agents and with people.

Despite this promise, applications of LLM-mediated multi-agent systems remain limited in scope. Much published work demonstrates coordination in simulation, code generation, data analysis, or web-based tasks. Comparatively few systems operate in the physical world, where latency, safety, sensor noise, and the need for reliable perception and actuation raise the bar for deployment. Real environments also require robust state tracking, recovery from failure, and cost-aware inference pipelines, all of which make naively transplanting lab-scale LLM-MAS designs into practice difficult. As a result, there is a gap between the flexibility shown in controlled demonstrations and the reliability expected in retail, public-facing, and other high-variability settings.

Human-agent interaction is another bottleneck. Most LLM-based systems still rely on text chat, which is accessible but constrains turn-taking and situational awareness. Voice interfaces exist, yet many are built around a wake word or a push-to-talk button that forces users to manage the floor of conversation and explicitly signal when to speak. This interaction pattern interrupts the flow of dialogue, prevents natural overlaps and backchannels, and discourages spontaneous engagement. It also keeps agents reactive rather than proactive, since they wait passively for a trigger instead of monitoring context and offering timely assistance. To realize the potential of LLM-MAS in everyday spaces, interfaces need continuous listening with reliable endpointing, interruption handling, and barge-in, along with policies that let agents decide when to initiate contact, when to yield, and how to coordinate turns with other agents and with people. Moving from text-first, trigger-based exchanges to context-aware, spoken interaction is therefore central to making multi-agent systems useful and trustworthy in real environments.

Therefore, this project is aiming to contribute on:

- A new agent-human voice interaction paradigm that gives agents the proactive ability to initiate, manage, and sustain conversations (e.g., context-aware triggers, interruption-tolerant TTS, and natural turn-taking), rather than relying solely on user-initiated commands.
- A real-world deployment of a large lan-

guage model (LLM)-mediated multi-agent system (MAS) in an unmanned clothing-store scenario, demonstrating role-specialised agents, multi-agent handover, and end-to-end voice interaction under practical constraints.

- An end-to-end, real-world LLM-MAS tech stack—including hardware designs, embedded software, a centralised orchestration and monitoring server, and visual debugging utilities—that others can reuse to accelerate user studies and iterative upgrades of real-world LLM-MAS.

2 LITERATURE REVIEW

This section first traced the development of MAS applications and identified current application gaps; it then survey existing human-agent interaction methods and explains their adaptation to this project; Finally, context engineering and prompt engineering were introduced, which are emerging techniques for configuring LLMs to fit them into different application scenarios.

2.1 Evolution of MAS Applications

Before the advent of LLMs, multi-agent systems were largely designed under rule-based or symbolic reasoning representation. In those traditional MAS, coordination among agents was achieved through well-established standardised mechanisms such as the contract net protocol [29], blackboard system architectures [4], or game-theory formulations of multi-agent cooperation and competition [28]. A classic examples is the robotic soccer environment developed in the RoboCup initiative [12], where both the states of the field and the agents’ corresponding behaviors are defined in advance. These pre-defined mechanisms were the primary drivers of inter-agent cooperation, as such, constrained system’s adaptability to unforeseen contexts. Moreover, there is a lack of natural interaction and communication between humans and agents from these systems, as agents are limited by their predefined communication languages (e.g., KQML, FIPA-ACL) and domain-specific ontologies¹, which makes MAS deployed predominantly in specialized, restricted-access operational environments, such as warehouses, workshops and laboratories [27], outside the purview of daily public interaction.

Contemporary MAS application is undergoing a profound shift with the emergence of LLMs. Incorporation of LLMs endows agents with adaptive reasoning step, which can flexibly produce new knowledge and heuristics as context changes, obviating the hand-crafted rule sets used in classical cognitive architectures [30]. MAS can leverage the natural language² understanding, general-purpose reasoning, and generative capabilities of LLMs to enable richer forms of coordination and dialogue among agents and humans, which extending the application domains of MAS [10]. The existing applications of large language model-mediated multi-agent system (LLM-MAS) can

¹domain-specific ontologies: It refers to formal, machine-interpretable conceptual models for a particular application domain, which defining the domain’s entities, relations, and constraints to ensure semantic interoperability and consistent reasoning across agents.

²natural language: It refers to a human language, such as English, as opposed to a constructed language, an artificial language, a machine language, or the language of formal logic

be summarised into two main lines: problem-solving and world simulation [8]. Typical problem-solving LLM-MAS applications include software development platform such as ChatDev [22] to accelerate the development process or “multi-agent debate” application such as AI Co-Scientist [7] for scientific research to enhance factual accuracy and the reliability of reasoning. World simulation applications treat agents as “social roles” and observing their emergent behaviors and strategic interactions in a controllable sandbox [8], such as game world [16], market or society simulator [1, 21]. Furthermore, there are multiple well-established LLM-MAS frameworks (e.g., AutoGen [34], CAMEL [15], Dify [5]) available to help users fast customise their desired system.

Despite this progress, the majority of LLM-MAS are still confined to text-based collaboration [8]. The absence of perception-action consistency and a verifiable execution chain across images, voice, videos, and actions restricts their reliable implementation in robotic and real-world scenarios [8, 9, 18]. As a result, the applications of LLM-MAS that we have witnessed are predominantly limited to software platforms and simulators, with limited deployment in everyday, public-facing settings. Building on these observations, this project implement an LLM-MAS in a physical retail setting (i.e., unmanned clothing-store), integrating sensors, actuators and intelligent agents on IoT edge devices and coordinating them via a PC-based central server for inter-agent communication to complete the retail tasks autonomously.

2.2 Human-Agent Interaction

Human-agent interaction serves as the primary entry point into MAS workflows. Among its modalities, voice interaction has become integral to AI assistant applications [23, 26]. Voice interaction has been adopted to areas such as smart speakers (e.g., Amazon Echo), in-car assistants (e.g., Mercedes-Benz MBUX), and smart-home hubs (e.g., Google Nest Hub). Their voice interaction designs demonstrate several common characteristics, such as hot-word triggering, turn-taking, and multi-round discourse.

Hot-word triggering (e.g., “Hey Siri” on Apple devices) remains the most common method for initiating conversations from user to AI assistants. The AI assistants continuously sample at low-power microphone path so a designated keyword can awaken the assistant at any time with limited computational overhead [11]. However, hot-word recognition can yield both false wake-ups and missed activations in a noisy public space (e.g., clothing-store), undermining reliable deployment of LLM-MAS in retail settings. In addition, hot-word triggering is initiated by users and

lacks the proactivity of agents to interact with humans. This is a drawback in the application of shopping scenarios. A further limitation of hot-word triggering is its reliance on user’s prior knowledge of the wake-up phrase, which is an unrealistic expectation in shopping scenario. Even if known, the voice recognition fails due to noise or other factors may cause socially awkward to user for ”talking to the void”, which can rapidly erodes user’s engagement and interest in the product. In order to make voice interaction compatible with the agents inside the unmanned clothing store setting, this project apply sensor technologies in the design of individual agent, enabling users to initiate conversations with the agent naturally, meantime ensuring low-computing state to saving power when no human appear.

Turn-talking is another feature of human-agent voice dialogue, where the user and the agent take turns to speak. Traditional voice interaction systems mostly adopt a half-duplex communication mode [2], where the end of speaking from user is determined by pressing a button (e.g., Textile-bot [35]) or a long period of silence (e.g., old-version Apple Siri). This approach cause delays and subsequently affect the interaction experience, especially prominent in a shopping scenario. To enhance the interaction experience, voice activity detection (VAD) has been introduced for natural turn management. This technique can determine the start and end of speech in-time, thereby more flexibly controlling the pace of the conversation [31]. In the human-agent voice interaction design for this project, the VAD technique was adopted, and a self-developed interruption function module was added, enabling users to interrupt the agent while it is speaking and continue the conversation smoothly afterwards.

Multi-round dialogue concerns that a voice agent can hold history context and carry out sequential conversations for several rounds of back-and-forth information exchanges, not just single question-and-answer sessions [24]. In unmanned clothing-stores, users’ needs can rarely be satisfied with one sentence alone. It needs to go through a series of clarification, refinement of needs, and progressive fulfillment of tasks. This needs to endow the voice system with remembering previous conversations’ content for several rounds. Therefore, context engineering is explored next to fill the gap.

2.3 LLMs with Context Engineering

Context engineering is an emerging topic which concerns how an LLM is conditioned at inference time

to reduce the hallucinations ³ [19]. It makes sure the output generation is grounded, stateful, and tool-aware. In practice, this shifts LLM’s capability from a purely parametric model to a context-augmented decision process. Representative, widely-used techniques include: (i) Retrieval-Augmented Generation (RAG), which fetches task-relevant evidence to ground outputs and mitigate hallucinations [14]; (ii) tool use via function/API calling, exemplified by Toolformer, which lets models decide when and how to call calculators, search, or planners and fold results back into context [25]; and (iii) external/long-term memory, which stores and retrieves episodic facts, plans, and interaction history across turns (e.g., Generative Agents) [20]. For LLM-MAS, these mechanisms can provide a shared, query-able substrate for collaborative dialogue, maintain global state for task allocation, and couple reasoning to perception/actions. This project does not invent a new contextual strategy; rather, it assemble a lightweight strategy which combines a compact per-agent memory and a minimal tool API to reduce the output hallucinations, with low adaptation cost and shorten the development cycle.

2.4 LLMs with Prompt Engineering

LLMs have shifted natural language processing (NLP) from a ”feature/architecture-centric” supervised learning paradigm to a ”pre-train and fine-tune” paradigm and, more recently, catalyzed a ”pre-train-prompt paradigm” [3, 17, 35], in which natural-language prompts steer (instruction-tuned) foundation models at inference time. This paradigm enables runtime task alignment without parameter updates or task-specific retraining, thereby reducing additional labeled-data needs and adaptation compute and improving the scalability of agent deployment across tasks. Accordingly, the prospects for applying LLM-mediated multi-agent systems (LLM-MAS) in everyday settings are increasingly feasible, though practical deployment remains subject to constraints such as infrastructure cost, communication latency, safety, and privacy [9].

To align the broad reasoning capabilities of LLMs with application-level behavior and reduce hallucinations, prompt engineering has emerged as a key methodology that aims to design efficient prompts for steering foundation models and guiding them to perform diverse downstream tasks [3, 35]. Recently, researchers have developed a variety of prompt designs: (i) instruction/role/specification prompts that align

³hallucinations: It refers to the cases that LLMs’ outputs that are fluent and confident yet ungrounded. This means outputs are factually incorrect or inconsistent with the given inputs or verifiable external evidence.

model behaviour and output format with task requirements [17]; (ii) reasoning scaffolds such as Chain-of-Thought and its zero-shot variant to elicit stepwise inference [33, 13]; (iii) task decomposition schemes including Least-to-Most and Plan-and-Solve that separate sub-goal planning from execution [36, 32]. In multi-agent settings, there is role-playing/inception prompting stabilises turn-taking, hand-offs, and division of labour to agents (e.g., CAMEL framework [15]). This project reuses and trim these existing prompting techniques into a lightweight prompt stack. This keeps adaptation cost low and reduce the development cycle for the system prototype, while laying a clean baseline for subsequent incorporation of more powerful prompting strategies in the future work.

3 TALKING GARMENTS PLUS MULTI-AGENT SYSTEM



Figure 1: TGP-MAS prototype deployed within a mini unmanned clothing-store setting - The blue mat marked with arrow at the bottom of the picture is the starting/initial point for Alice and Jarvis. The top left corner red mat is where Garment Agent Nova placed, and the top right corner red mat is where Garment Agent Bob placed.

This section introduces a novel application instance of LLM-MAS that has been developed in this project. Section 3.1 first justifies the choice of unmanned clothing-store setting by analysing the limitations of prevailing unmanned retail examples and outlining how an unmanned clothing-store can contribute to sustainable development objectives. Accordingly, Talking Garments Plus Multi-agent System (TGP-MAS) is introduced (see prototype in Figure 1), presenting its system architecture overview (see Figure 2) and the

system workflow (see Figure 3) that traces the end-to-end shopping experience supplied from the system to users.

3.1 Unmanned Clothing-Store

Unmanned retail has moved from concept to practice with “checkout-free” formats that combine sensor fusion and computer vision to streamline in-store flows (e.g., Amazon Go). This “checkout-free” formats of unmanned retail such as Amazon Go work well for daily “grab-and-go” grocery shopping, which doesn’t demand much consultation. By contrast, garment belongs to a high-involvement category ⁴ in which consumers must resolve attribute uncertainty (e.g., fit/size, fabric composition, country of origin) and often seek recommendation from sale associates. An LLM-mediated multi-agent system can deliver on-demand product knowledge (e.g., size reconciliation, fabric explanations), conversational styling, and real-time coordination among agents, thereby retaining the experiential benefits ⁵ from shopping while preserving automation.

As a complementary objective, the unmanned clothing-store setting is chosen to align with and contribute to UN SDG 12 (Responsible Consumption and Production) ⁶. The textile industry is a significant source to global carbon emissions, with estimates suggesting it contributes about 10% of global emissions, similar to the sum from international air travel and seaborne transport [6]. Meanwhile, human sale associates typically retain only salient product basics (e.g., size, fit, price), while information such as garment fabric and sustainability literacy is chronically under-supplied at the point of sale. Recently, the TextileBot [35] demonstrates the feasibility of LLM-mediated agent (LLM-Agent) specialise in textile circularity domain through prompt engineering without task-specific model training or fine-tuning. It provides insights for applying LLM-Agent in unmanned clothing-store to bridge these knowledge gap, make them capable to advocate responsible consumption at the point of sale.

⁴high-involvement category: It refers to products or services that consumers perceive as significant, complex, and risky, requiring extensive time, effort, and research before a purchase is made.

⁵experiential benefits: It refers to the positive outcomes from actively participating in an experience, leading to stronger emotional connections, increased learning, and memorable outcomes.

⁶UN SDG: The United Nation Sustainable Development Goals (SDGs) are a universal framework of 17 interconnected goals adopted by the United Nations in 2015 to create a better, more sustainable future by 2030

3.2 TGP-MAS

Figure 2 depicts the system architecture overview of TGP-MAS, highlighting the core components and their data/interaction flows:

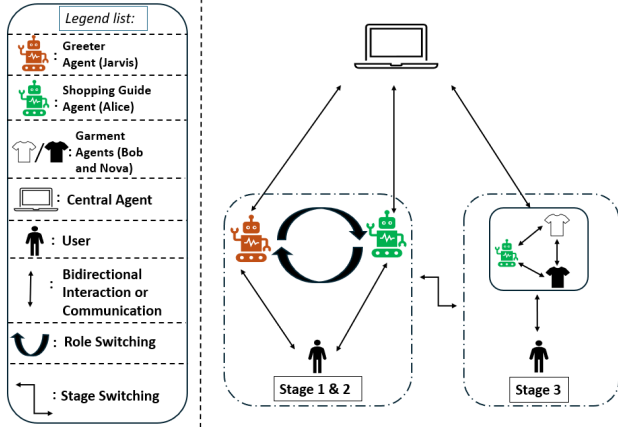


Figure 2: TGP-MAS architecture overview - The legend list marks all the relevant graphic symbols. The overview of the system architecture is on the right of the dotted line.

As illustrated in Figure 2, TGP-MAS comprises five agents and operates across three stages. Greeter (Jarvis) and shopping-guider (Alice) co-reside on a Raspberry Pi 4 Model B micro-processor, which is mounted on a TurtleBot3 Burger mobile base. They operate in Stage 1 and Stage 2 respectively and handing over control as the system transitions between stages. In these stages, users engage in turn-taking dialogue with each agent. Alice will lead the user into Stage 3 and activate the corresponding garment(s) (Bob or Nova or both) to start the multi-agent-human dialogue. Central agent exists in the back-end⁷ and plays a crucial role in the multi-agent-human dialogue. It records all the completed conversations among user and agents. The chat-history are subsequently fed into the LLM as one of the inputs to generate the outcome for selecting the next suitable speaker in the dialogue. Central agent is also responsible for agents’ registration and monitoring their real-time status, enabling agents to know others’ presence.

Figure 3 explains the complete operation workflow of TGP-MAS. Starting from the "Initial State" in Stage 1, the workflow proceeds step by step in the direction of the black arrows. The rectangular area represents ongoing activities. The rhombus area is the fork point of the system workflow, and the system workflow will choose which branch to follow based on the current condition. The green double-headed arrow represents that the central agent participates in the on-

⁷back-end: the part of a computer system or application that is not directly accessed by the user

going activities of the connected rectangular area. The red arrow is the direct path for the system overflow return to the starting point, which indicates the system reset. Circular areas exist in parallel with all activities at each stage and are used to handle exceptional circumstances, such as when a user has not responded for a long time or has clearly indicated a desire to leave without completing all expected interactions until the "End" condition in Stage 3.

The following sub-sections elaborate on an expected complete interaction workflow between TGP-MAS and users (illustration version of workflow in Figure 14 in Appendix section A), which can be summarised into three stages and the parallel exceptional circumstances handling logic:

3.2.1 Stage 1: find customer

The main task in Stage 1 is to identify passersby and gain their attention, thereby turning them into customers of the clothing-store. Starting from an idle initial state, Jarvis joins the workflow and continuously performs human detection algorithms. When a passerby is detected, Jarvis initiates a brief greeting and asks whether he/she would like a free guide to explore the unmanned clothing-store. If the user declines, gives no response, or indicates they wish to leave, the session ends without escalation. If the user consents, Jarvis triggers a handover to Alice through role switching.

3.2.2 Stage 2: guide customer

After role switching, Alice begins by welcoming and confirming the customer’s intent. If the customer has unknown interest, Alice continues the conversation, eliciting customer’s preferences (e.g., category, style, budget, sustainability concerns) until a candidate direction emerges. Once the customer’s interest be found out, Alice submits a request to the Central Agent for path planning to the relevant area where the garments are placed. When the plan is returned, Alice moves and leads the customer to the target location, marking arrival as a new stage.

3.2.3 Stage 3: customer join conversation with garments

Upon arrival at the target area, the workflow enters Stage 3, where customer-garment interaction takes place. Alice allows the customer to explore freely. When a garment of interest is identified, the customer may either inform Alice or interact with the garment directly (e.g., touch or pick it up); either action "wake-up" the embedded Garment Agent, which then initiates the conversation. The Garment Agent presents

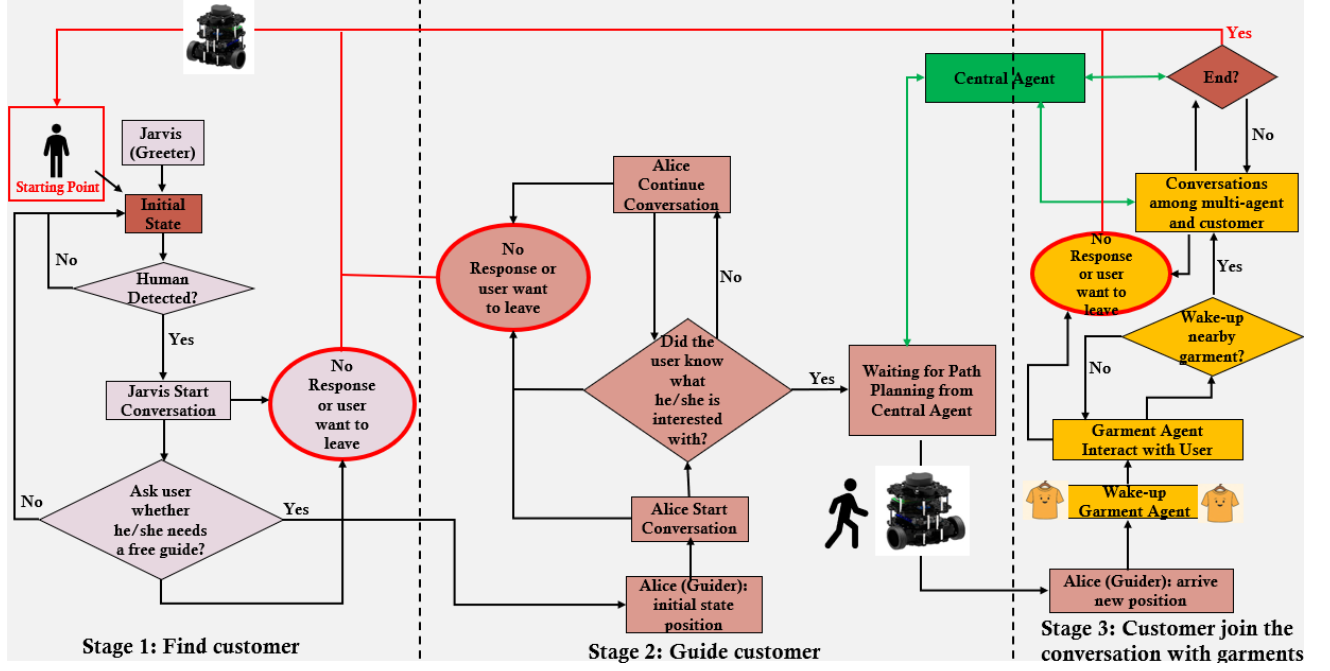


Figure 3: TGP-MAS operation workflow - This figure uses four colors (from left to right: lavender, pink, green and yellow) to distinguish the stages involving Greeter, Shopping-Guider, Central and Garment Agent(s). The two deep red status, "Initial State" and "End?", respectively represent the starting point and the end point of the system workflow.

itself (e.g., fit, fabric, sustainability story, care instructions) and answers product-specific questions; Alice moderates, provides store-level information (e.g., size availability, pricing, promotions). If appropriate, additional nearby garments may be remotely activated by Alice to "join" the conversation to offer contrasts (e.g., lighter fabric vs. better durability), creating the multi-agent conversations in the format of debate. This enables users to gather information across garment styles and make an informed final choice through comparison, rather than being constrained by the items that initially attracted their interest. To prevent unbounded garment wake-ups and comparison loops, the Central Agent monitors dialogue state at server-side and schedules closure at appropriate moments, for example, when the user's stated budget has been reached or preferences have converged.

3.2.4 Exceptional circumstances handling

To address the situation where a user does not respond, clearly indicates a desire to leave, or technical errors occur (e.g., a broken microphone) before the interaction workflow reaches the final "End" condition, the system has established the parallel exceptional circumstances handling mechanisms at each stage:

When the user explicitly or implicitly indicates their intention to leave before complete all expected interaction (i.e., reach "End" condition), the ongoing agent

will engage in a final farewell conversation with the user and the system return to the initial state subsequently. For handling cases when no response received from the user, each agent has set a customised time threshold (currently set at 7 seconds). When the first time threshold is reached, the agent will proactively prompt with a message such as "Are you still there?" If the user does not respond by the second time threshold, the system will directly return to the initial state of Stage 1. If the user responds before the second time threshold is reached and indicates a desire to think for a moment, the agent will inform the user of the hot-word to resume the conversation (e.g., "Hey, Nova"). There will be a one-minute time threshold to wait for the user to use the hot-word to reactivate the conversation; if it times out, the system return to the initial state without further notification.

4 METHODOLOGY

This section focuses on elaborating the methodology of TGP-MAS. It first describes the approach for enabling voice interaction between users and LLM-Agents, then proceeds to introduce the design of each agent from the perspectives of hardware and software. Finally, it explains the LLM usage on agents to make them intelligent and capable to deal with the tasks on unmanned clothing-store setting.

4.1 Voice-Interaction with LLMs (Unified Across All Agents)

LLM-enabled applications primarily facilitate users' interaction through text input/output (e.g., ChatGPT). To enable users to communicate with embedded LLM-Agents via voice-interaction, this project has designed a unified module to convert users' voice input into text for LLM processing and transform the LLM-generated results back into voice to achieve bi-directional voice-chat between users and agents.

As shown in Figure 4, the voice-interaction with LLMs component of TPG-MAS is constructed with a loop architecture across six core stages: (i) capture user audio (through microphone) (ii) detect utterance start/end via WebRTC VAD, (iii) transcribe speech to text via Speech-to-Text technique (STT), (iv) query an LLM using transcribed speech as input to generate response in text, (v) render the LLM response from text to MP3 file via Text-to-Speech technique (TTS) and play the audio (through speaker), and (vi) Hot-word barge-in to disrupt agents while speaking. The following subsections align with Figure 4 to elaborate on the design of sub-modules which combined together build up the voice-interaction component for TGP-MAS:

4.1.1 Parameters and deployment notes

Here are core tunables for configuring voice-interaction with LLMs component: sample rate (16 kHz), frame size (30 ms), VAD modes (2 for end-pointing; 0 for barge-in monitor), min/max utterance: (0.5/20 s), silence end-pointer (0.8 s) and keyword list ("stop talking"). These configurations are module constants so all agents behave identically in terms of the voice-interaction methodology, while remaining site-tunable for improvement after user evaluation. The TTS path avoids external players (pure Python I/O) and the monitor thread is short-lived per utterance to keep resource usage predictable on Raspberry Pi-class hardware.

4.1.2 Microphone: audio-in

An agent begins by opening a low-latency input stream on the default microphone. Audio is captured as 16-bit mono PCM⁸ at 16 kHz in 30 ms frames with **blocksize = FRAME_SIZE = 16 000 × 30 ms = 480 samples**. Each 30-ms block is pushed into a queue and synchronously scored by WebRTC VAD. These constants are shared across agents so that downstream

⁸PCM (pulse-code modulation): Uncompressed digital audio in which each sample stores the instantaneous amplitude of the signal. In this project, *PCM* means 16-bit signed little-endian, mono, 16 kHz audio, processed in 30 ms (480-sample) blocks for VAD/STT/TTS. It carries no headers or metadata—it is just the raw stream of samples.

modules (VAD, STT, and the TTS monitor) operate on the same temporal resolution.

4.1.3 Voice activity detection (VAD): detect start/end-of-speech

This project implements endpointing⁹ with WebRTC VAD on fixed-length audio frames, combined with simple temporal thresholds. The goal is to (i) robustly detect when a user starts speaking, (ii) return a complete utterance as soon as natural speech ends, and (iii) keep latency predictable on embedded hardware.

For routinely listening from agent to user, this component set VAD mode = 2 ("balanced aggressiveness"), which reduces false positives in retail acoustics while remaining responsive to conversational speech. Three variables are tracked: a Boolean "*speaking*", a byte buffer "*pcm_buffer*", and a trailing-silence accumulator "*silence_time*" (in seconds).

When VAD first flags speech on incoming frames, the component switch to "*speaking=True*" and begin buffering frames via adjusting on "*pcm_buffer*". While "*speaking=True*", non-speech frames increment "*silence_time*" by 0.03 s per frame. When trailing "*silence_time*" ≥ 0.8 s, the component finalise the utterance and return the buffered PCM marking the end-of-speech. Two guards bound quality and duration: segments < 0.5 s are discarded as spurious; segments ≥ 20 s are force-ended to avoid run-on monologues. These thresholds stabilise turn-taking and keep end-pointer latency under one second.

4.1.4 Speech-to-Text (STT)

When end-of-speech is detected, the buffered PCM is exported to a temporary WAV¹⁰ and transcribed with Whisper (whisper-1). Transcription failures (e.g., due to sudden network disconnected) return an empty string (the loop resumes listening) and sub-second clips are ignored to avoid false triggers. This turn-level STT is identical for all agents, producing normalised text for the multi-agent dialogue design.

4.1.5 LLM inquiry (GPT-4o-mini) : generate response in text

The transcript is appended to a bounded chat history (system prompt + last 10 messages) and sent to the Chat Completions API (model="gpt-4o-mini")

⁹endpointing: In speech interfaces, endpointing is the process of detecting the *start* and *end* of a user's utterance so the system knows when to begin and stop buffering audio for transcription.

¹⁰WAV: A container file format that wraps audio data—typically PCM—with a small RIFF header that specifies sample rate, bit depth, and channel count. In this project, it writes temporary WAV files (16 kHz, mono, 16-bit PCM) as a convenient wrapper accepted by transcription libraries; the underlying audio content is the same PCM described above.

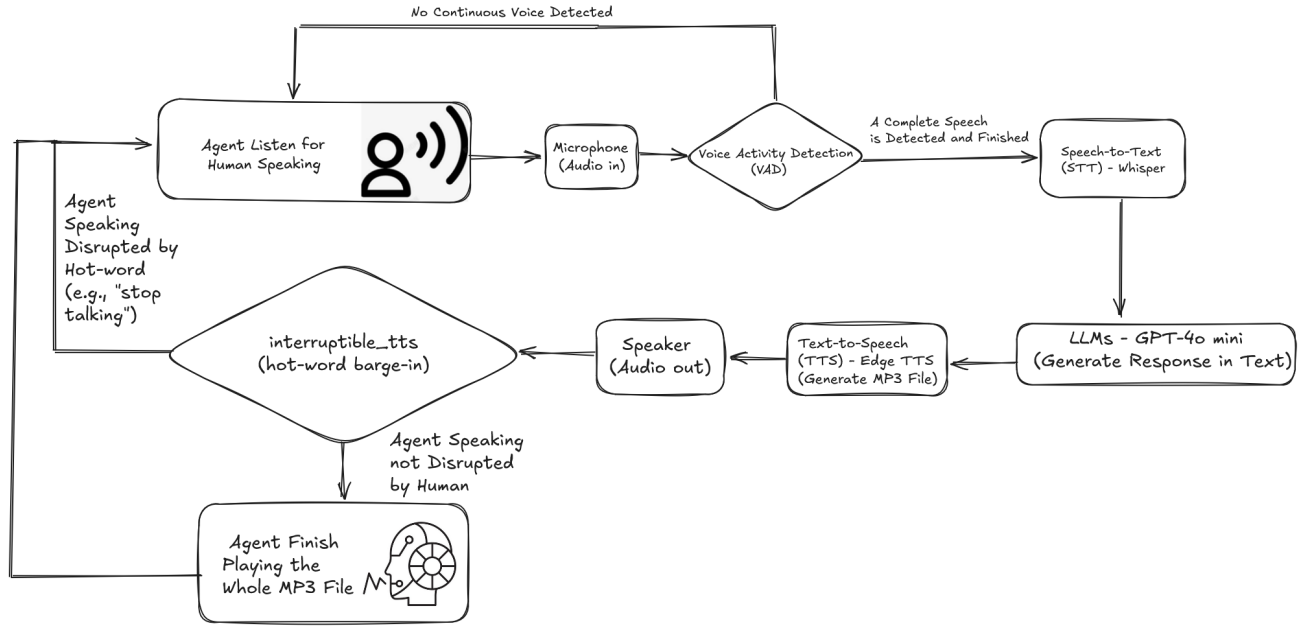


Figure 4: Voice-Interaction with LLMs. To facilitate understanding of this workflow, it’s recommended to take the “Agent Listen for Human Speaking” in the upper left corner of the diagram as the starting point and proceed step by step along the arrows. The rectangular areas in the diagram represent ongoing activities at each step, while the rhombus areas indicate condition checks to determine the next step

to generate a concise reply. To limit repetition in a retail setting, replies are checked against a local storage of response history and regenerated up to three times if a duplicate is detected; a lightweight intent gate is designed to asks the model to answer YES/NO on whether the user is determined to leave based on semantic comprehension, enabling graceful closes for the exceptional circumstance as described in section 3.2.4. More content about LLM response design is covered in intelligent agent design section.

4.1.6 Text-to-Speech (TTS) and speaker: audio out

Text responses from LLM are synthesized with Edge-TTS into an MP3, decoded to 16 kHz mono PCM via pydub, and streamed to the speaker with the class *sounddevice.RawOutputStream* from Python package in 30 ms chunks. A per-chunk RMS level callback exposes dBFS levels (-120..0 dBFS)¹¹ for driving synchronous OLED visual feedback while speaking in the garment agent design, more details about the synchronous OLED visual feedback is covered in agent design section (Hardware Design). Audio playback is governed by thread-safe events so agent speech can be stopped via hot-word without clicking on physical interfaces. These events return a boolean indicating

¹¹dBFS (decibels relative to full scale): It is the most commonly used level unit in digital audio. “-120..0 dBFS” is an abbreviation for “from -120 to 0 decibels”

whether playback was interrupted.

4.1.7 Interruptible_tts (the hot-word barge-in mechanism): disrupt agent while it is speaking

While the agent is speaking, a parallel monitor opens a second microphone stream and runs WebRTC VAD at mode 0 (permissive) so user speech is not missed. Two interruption modes are designed and are used depending on the microphone technology integrated:

- **Hot-word mode (default).** The monitor buffers a short segment, treats 0.4 s of silence as segment end, transcribes the segment with Whisper, and stops playback if the text contains any configured hot-word (e.g., “stop talking”). A 0.5 s start delay reduces self-echo. This mode was eventually applied to this project because it uses a USB-microphone for environmental sound detection.
- **Fallback mode.** If no keywords are configured, the monitor stops playback after ≈ 1.3 s of continuous speech (this parameter is adjustable for finding the suitable value after user evaluation). This mode is compatible with directional microphone (e.g., Lexinova AY-321) or background noise canceling microphone (e.g., Lavelia). Because environmental noise (e.g., the echo from agent itself) can cause a false interruption while the agent is speaking.

If no hot-word is detected and the user remains silent while agent is speaking, the agent completes playback of the synthesised MP3, send end-of-speech status to the monitor thread, and immediately re-enters the listening state. This tail-path is implemented by marking the player “done” and joining the monitor briefly to exit cleanly.

4.1.8 End-to-end voice interaction sequence (shared across all agents).

These sub-modules in combination deliver a fast, robust, and consistent voice interface across agents with the following end-to-end voice interaction sequence: Agent listen to human speech with VAD $\rightarrow$ detect utterance end (≤ 0.8 s silence or 20 s cap) $\rightarrow$ transcribe voice with Whisper STT $\rightarrow$ build bounded context and query gpt-4o-mini for response in text $\rightarrow$ Agent speak the text via interruptible TTS (monitor keywords) $\rightarrow$ if interrupted, stop and return to listening; else finish playback and loop. If no speech appears within 7 s, issue a short reminder, listen again, and, if still silent, deliver a polite goodbye and exit. All steps emit structured logs and heartbeats for system-wide coordination.

4.1.9 Rationale and implications

Using mode 2 for turn-taking prioritises stable end-points and natural pauses; mode 0 for barge-in prioritises sensitivity so users can reliably cut in. A 0.8 s trailing-silence threshold balances snappy returns with protection against mid-phrase pauses. The 0.5–20 s bounds filter accidental triggers and cap monopolizing monologues. Finally, aligning both channels to 30-ms frames simplifies implementation and avoids clock-skew artifacts between listening and interruption paths. Together these choices yield predictable, low-latency endpointing that feels natural in a multi-agent retail environment

4.2 Hardware Design

This section expanding the IoT hardware techniques employed in designing the physical intelligent agents for TGP-MAS, including embedded system design and sensor integrations. In addition, this section also introduces the physical body design of each agent.

4.2.1 Greeter (Jarvis) and shopping-guide agent (Alice) embedded system design

This project selects the Raspberry Pi 4 Model B as the micro-processor for Jarvis and Alice design because it meets four core requirements: (i) reliable real-time I/O for speech and vision, (ii) low power and cost at scale, and (iii) a mature Linux/Python stack that shortens

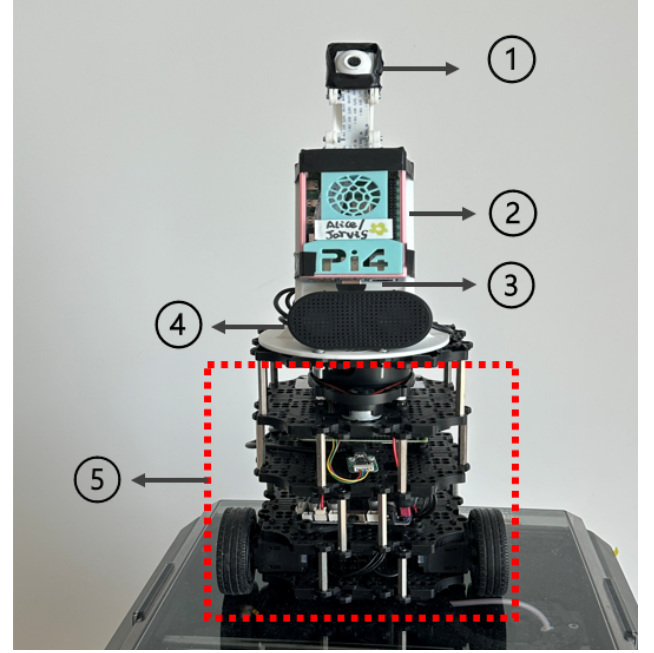


Figure 5: The Physical Body of Jarvis and Alice - The hardware used from top to bottom are :(1) Raspberry Pi Camera V2.1, (2) Raspberyy Pi 4 Model B micro-processor, (3) Garnirise Mini USB Microphone, (4) UKHONK Mini USB Speaker and (5) TurtleBot3 Burger.

integration time,(iv) same micro-processor model with the one for driving TurtleBot3 Burger, which can reduce the incompatibility issue intuitively. On-device pre-processing (voice activity detection, endpointing, and audio buffering) runs deterministically at 16 kHz with 30 ms frames and 0.8 s silence thresholds, providing stable latency budgets before cloud STT/LLM calls; the current implementation uses webrtcvad with a uniform audio pipeline across agents. The same platform also supports Raspberry Pi camera-based presence sensing and face-mesh tracking for proactive greeting, implemented with Picamera2 and MediaPipe on Raspberry Pi OS.

Although Jarvis (greeter) and Alice (shopping-guide) have distinct personas and policies, they share the same sensing and actuation stack: camera presence detection, far-field microphone capture, low-latency TTS output, the identical VAD/endpointing routine (16 kHz, 30 ms frames, 0.8 s trailing silence) and carrier (TutrlBot3 Burger). Consolidating both agents on a single Pi eliminates cross-device network hops during handover, reduces synchronization complexity, and allows the two roles to time-share the audio devices without arbitration. In practice, Jarvis initiates a conversation (human-presence trigger), starts a conversation record at the hub, and handover the active conversation_id to Alice upon a positive user intent;

the transfer is a local process switch that avoids context loss (i.e., chat-history) and round-trip delay, this makes the process from attracting customers to taking users to explore the store seamless, meantime reduce additional carrier cost.

This co-location also improves robustness and maintainability. A single watchdog and logging stack supervises both processes; if one role restarts, the other can continue emitting heartbeats and preserving telemetry. Having a shared */tmp* state and a common *central_client* contract ensures that interruptions (e.g., the “stop talking” keyword) and time-outs are recorded consistently across roles, while the same *speak_and_listen* interface (the function used in voice-interaction component) keeps the TTS pipeline and interruption semantics uniform. The result is a simpler bill of materials, lower energy draw, and measurably lower handover latency, without sacrificing modularity at the software level (each role keeps its own prompts, reply policies, and dialogue memory while adhering to the same device and observability APIs).

4.2.2 Garment agents (Nova and Bob) embedded system design

The garment agent is the embedded conversational, object-centric agent that represents the product (i.e., clothing) in the store. The embedded system circuit design for garment agent is shown in Figure 6. As shown in Figure 6, garment agent integrates inertial sensing, capacitive touch and remote activation from server for multi-modal wake-up. It also uses a 128×64 SH1106 OLED for expressive visual feedback and integrates the voice interaction pipeline as described in section 4.1. Together these components enable both active and passive interaction with garment agent from the user. Below details the hardware design of the garment agent. The explanation proceeds from the choice of microprocessor to the sensor stack, and finally to the OLED-based agent’s visual expression display.

Starting from the microprocessor choice. The garment agent must deliver always-on wake interactions, low-latency voice I/O, and lightweight on-device control while remaining cost-sensitive for per-garment deployment. The initial plan adopted the Raspberry Pi Zero 2 because its price point makes large-scale embedding across multiple garments economically viable. However, the earlier developed voice interaction component for the system requires two simultaneous USB endpoints (e.g., for independent microphone and audio output paths / audio interface), and the Zero-class board’s limited USB connectivity proved constraining. Therefore, the project finally selected the Raspberry Pi 3 Model B+, which balances bill-of-materials cost

against significantly higher computing and I/O headroom. Compared with Raspberry Pi 4, the Pi 3B+ reduces cost and thermal load while still providing sufficient CPU performance and enough USB interfaces to support the dual-USB voice pipeline alongside the I²C sensor bus. This trade-off enables an affordable, replicable garment-level micro-node that can be scaled across a rack of items without sacrificing responsiveness in speech interaction and the visual feedback from OLEDs.

Shifting to the sensor/actuator suite integration and multi-modal activation mechanism for garment agent. The garment agent integrates two complementary physical user-activation pathways, (1) “pickup” and (2) “touch”, these two ways are implemented with an inertial measurement unit (IMU) and a capacitive touch sensor respectively. The reasons for designing these two types of interactions are: (1) When users clearly know the clothes they are interested in, they usually pick them up directly to try on or check them out. At this time, the agent needs to be awakened quickly to avoid missing interaction with user. (2) When users are just browsing, they usually randomly touch the clothes to feel their materials. At this time, the agent can be awakened slowly. If a user touches the garment for a long time, it usually indicates that they have become interested in it. This is the best time to activate the agent, and it can also prevent the garment agent from being overly activated at unnecessary times, thus saving the power resource.

The IMU is an MPU-6050 connected over I²C. Wiring follows the Pi’s default I²C pins: VCC→3.3 V, GND→GND, SDA→GPIO2 (Pin 3), SCL→GPIO3 (Pin 5). The chip address is determined by AD0: tying AD0 to GND selects 0x68; tying AD0 to 3.3 V selects 0x69, allowing address flexibility if multiple devices share the bus. The firmware configures the device and then continuously samples accelerometer and gyroscope data, declaring a “pickup” event when the acceleration magnitude deviates from 1 g by more than 0.20 g or when the summed angular rate exceeds 40 °/s for three consecutive samples (with a 50 ms polling interval) and with a short confirmation read to reduce false positives. This design makes the garment “wake” naturally when a shopper lifts or handles it, without requiring explicit button presses.

A TTP223B capacitive touch pad provides a second, intentional activation channel. It is wired VCC→3.3 V, GND→GND, and OUT→BCM17 (Pin 11) and, in our configuration, produces an active-high output. Debounce (120 ms) and a short cooldown (0.6 s) suppress spurious activations in retail settings. To differentiate casual taps from deliberate user intent, the garment agent uses a 3.0 s long-press as the definitive “wake” action; during this press, the on-garment

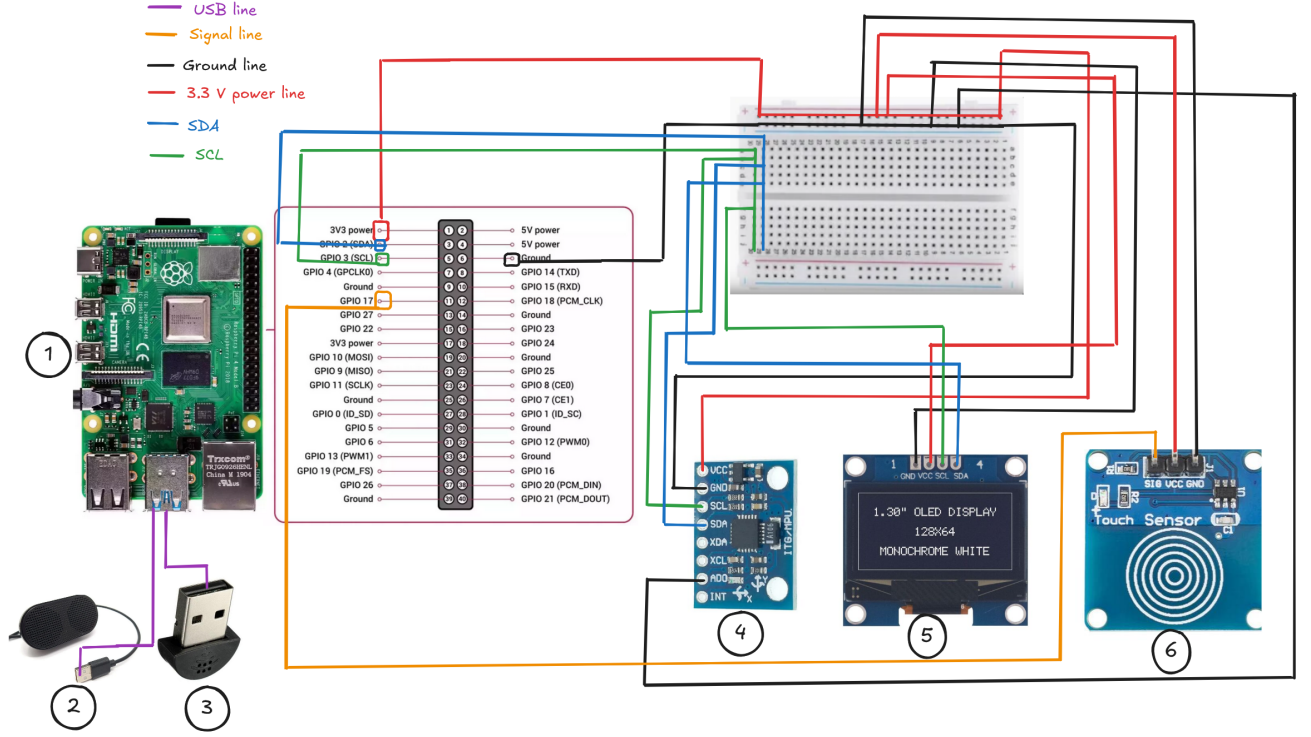


Figure 6: Garment Agent Embedded System Circuit Design - The hardware used from left to right are: (1) Raspberry Pi 3 Model B+ micro-processor, (2) UKHONK Mini USB Speaker, (3) IMU module MPU-6050, (4) IMU module MPU-6050, (5) SH1106 OLED (128×64) and (6) TTP223B capacitive touch sensor. The breadboard is used in the figure for better visualisation of the pin and wires connections among different modules.

OLED executes a progressive eye-opening animation, and releasing before the threshold smoothly reverses the animation back to sleep. This temporal thresholding provides a low-friction but robust interaction grammar that avoids accidental wake-ups while remaining accessible to users who expect touch affordances on smart products.

The distinct feature of garment agent compared to greeter and shopping-guider is the OLED display and expressive state machine. A SH1106 OLED (128×64) on the same I²C bus provides a minimalist, low-power “face” for the garment. The driver probes standard SH1106 addresses (0x3C/0x3D) and initializes the display with a neutral contrast setting; the shared bus arrangement with the IMU is safe because the addresses are distinct and the initialization logic discovers the active address at runtime, improving field robustness. The display runs an expression library that encodes agent states as legible micro-animations: sleep (closed eyes with gentle “Z” drift), listening (open eyes and symmetric side-wave arcs), speaking (animated mouth cycles), and thinking (smiling eyes with a looping ellipsis). Additional affective variants—happy, surprised, angry, and wink—are used contextually for lightweight feedback. During text-to-speech, the system feeds

real-time audio level callbacks into the renderer so that pupil offsets and mouth shapes track voice energy, giving a subtle sense that the garment is “speaking.” When the agent is muted (via a semantic voice command), the OLED switches to a “muted listening” motif (mouth as an “X”, small head-shake, and teardrop markers) to acknowledge the state without audio. These display states both humanize the interaction and externalize internal modes (idle, wake, listen, speak, think, mute) so that bystanders can immediately infer whether the garment is ready to engage.

Other crucial embedded system design considerations of garment agent are: I²C, power, and wiring. The agent keeps all low-voltage peripherals on the Pi’s 3.3 V rail to simplify power distribution and protect logic-level compatibility: the SH1106 OLED and MPU-6050 both take 3.3 V VCC; SDA/SCL use the Raspberry Pi’s native pull-ups on GPIO2/GPIO3. The IMU and OLED occupy different I²C addresses (as noted above), so no multiplexing is required. The firmware includes a small utility to scan available /dev/i2c-* nodes and select a working bus dynamically before bringing up the sensors and display, which reduces configuration drift across boards. This approach lets multiple garment agents be flashed from a common

image and deployed without per-unit address fiddling.

Putting it together in use: In the idle state the OLED shows a sleeping face. A shopper either (a) lifts the garment, satisfying the IMU motion thresholds and thereby triggering wake, or (b) performs a deliberate long-press on the touch pad; both paths transition the face through a progressive eye-open animation into the listening state. Once listening, voice endpointing and turn-taking are managed by the system’s audio stack; when the agent speaks, the OLED’s mouth and pupils animate in synchrony with the synthesized speech, and on conversation end the eyes slowly close back to sleep. The simple wiring (two-wire I²C plus one GPIO input), the conservative thresholds (0.20 g / 40 °/s) with confirmation reads, and the long-press gesture together deliver a fault-tolerant, glanceable interface that feels native to a retail context—responsive when handled, restrained when ignored.

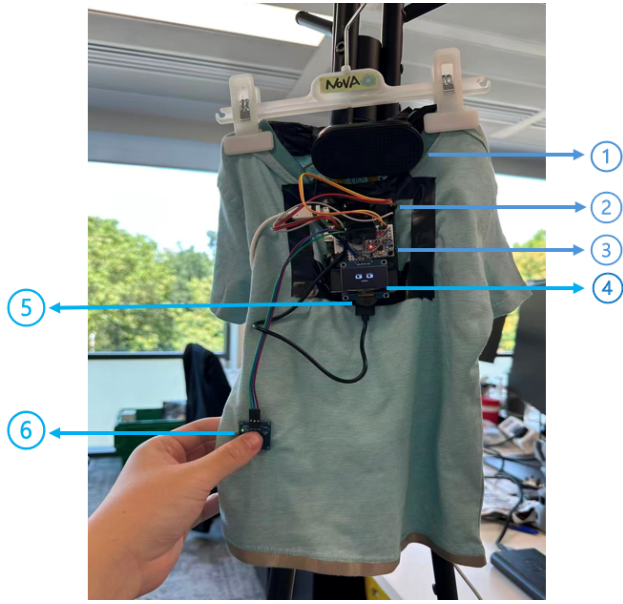


Figure 7: The Physical Body of Nova - The hardware used from top to bottom are : (1) UKHONK Mini USB Speaker, (2) Raspberry Pi 3 Model B+ micro-processor, (3) IMU module MPU-6050, (4) SH1106 OLED (128×64) (5) Garnirise Mini USB Microphone and (6) TTP223B capacitive touch sensor.

Figure 7 presents Nova, the first instantiated garment agent. Nova adopts a fully embodied design in which the garment itself is the agent: all sensing hardware and the microprocessor are permanently integrated into the fabric through stitching and concealed seams, so the agent is inseparable from the clothing item. While this configuration delivers a compelling sense of immediacy that users can interact directly with a “speaking” garment on the rack, but later this design is recognised with lifecycle limitation. Once a

shopper purchases the item, the value proposition of an on-garment conversational agent largely ends; most users will not continue wearing a talkative garment in everyday life, and the agent’s primary function is to augment the in-store browsing and decision-making experience. To address this constraint, the second garment agent instance, Bob, adopts a decoupled embodiment (see Figure 8): the agent becomes a detachable module that docks onto the hanger or neckline area and operates more like an in-store placard or smart tag. After the garment is sold, the module can be removed within seconds and reattached to unsold inventory, enabling rapid redeployment, amortization of hardware costs across multiple items, and a substantial reduction in e-waste. This separation also simplifies maintenance (cleaning, battery/service swaps) and supports consistent interaction quality across sizes and SKUs without committing electronics to any single piece of apparel.

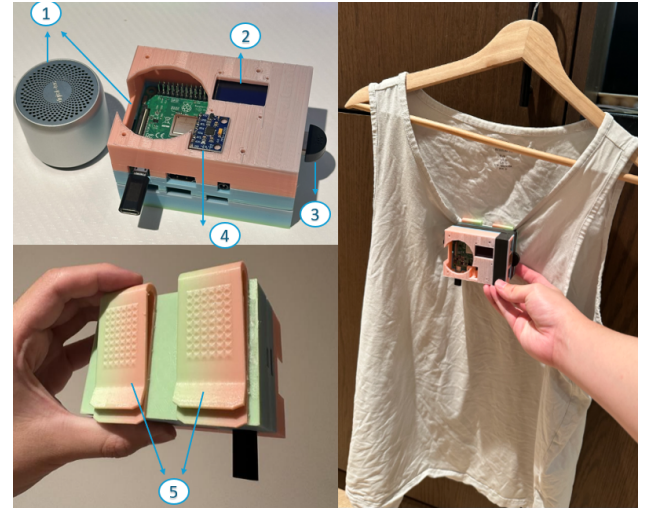


Figure 8: The Physical Body of Bob - The key design features of this body are : (1) IP67 Portable Bluetooth Speaker and its placed position on its body, (2) OLED position (3) USB-microphone position, (4) IMU MPU6050 position (5) Built-in clamps for easy installation/removal of agent into garment. The right picture shows the effect drawing of the garment after installing this device.

It is worth noticing that when installing Bob onto the garment, as shown in Figure 8, the IP67 Portable Bluetooth Speaker was missing, resulting in a circular gap at the position where this speaker was supposed to be placed. This was due to a millimeter-sized discrepancy between the 3D printing outcome and the design drawing, which prevented the speaker from being successfully installed into the model. Due to the tight schedule, the 3D model was not redesigned. Instead, the speaker was placed in the nearby area of the clothes

and connected via Bluetooth to enable Bob to speak. Additionally, it is worth noting that the model did not have a designated area for the TTP223B capacitive touch pad. This was because it was originally planned to use the Adafruit Capacitive Touch HAT along with the tape-like sensor to replace the TTP223B capacitive touch pad, as this sensor was more suitable for assembly onto clothes. However, due to technical complexity issues, the compatibility work between this sensor and the Raspberry Pi 3 Model B+ was not completed, resulting in neither of the two touch sensors being installed in Bob. Nevertheless, this does not affect the final test, as Bob still has system wake-up and pick-up wake-up methods from user.

4.3 Software Design

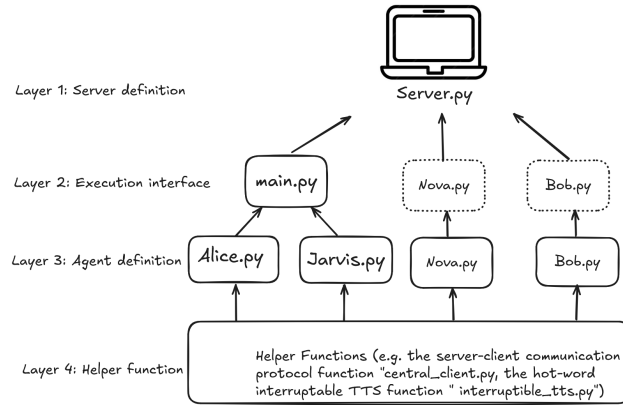


Figure 9: Software Hierarchical Structure of TPG-MAS - Due to the fact that different agents operate on different devices, it makes it difficult to run the entire system with a single click.

Therefore, the software-level design of this system adopts this hierarchical architecture, where the layers from the top to the bottom represent the corresponding file’s running priority, with the top layer having the highest priority. Note that currently, the execution interfaces of the garment agents Nova and Bob are in the same file as their own definition files. Future work will design execution interfaces for them as well, thereby distinguishing between running files and functional files, allowing users to modify and debug garments agent more conveniently.

The software design of TGP-MAS is a hierarchical structure (see Figure 9) that cleanly separates centralised agents management, agents profiling and agents interaction methods. Edge agents (greeter, shopping-guider and garment agents) run on Raspberry Pi devices and handle sensing and human-agent dialogue. A minimal “Central” service running on the PC/Laptop provides shared infrastructure for agent registration, liveness heartbeats, con-

versation life-cycle management, message/event logging, and cross-agent behaviors. This split keeps each agent’s runtime simple and resilient while preserving a single, queryable source of truth for evaluation and debugging across the entire user journey.

4.3.1 Server-client protocol design

At the heart of server’s coordination is a FastAPI application that exposes a small, explicit surface area. Typed request models define the contract for `/register`, `/heartbeat`, `/conversation/start`, `/message`, and `/handover`, and the server retains only what is required for a proof-of-concept: in-memory dictionaries for agents and conversations plus an append-only message log. When an agent registers or heartbeats, the service updates internal state and broadcasts the event to any connected dashboards; starting a conversation returns a globally unique *conversation_id* that agents then attach to subsequent logs. Messages (both user and assistant turns, as well as semantic “event” rows) are normalized via a single POST endpoint and enriched with a server-side timestamp before being broadcast and persisted in memory. The handover endpoint updates the participating agent set for a conversation and emits a dedicated event, making role transitions and multi-agent turn-taking observable at a glance. For quick inspection, the server also exposes read-only list endpoints—`/agents`, `/conversations`, `/messages`—and an ultra-light HTML dashboard that renders a live stream with role-aware styling. WebSocket broadcasting keeps this dashboard reactive, making the system’s state and traffic visible without pushing orchestration complexity into the agents themselves.

Edge agent processes talk to the Central service through a tiny client library that wraps HTTP calls and handles operational concerns by default. On startup, each agent registers its identity, declared type (e.g., “greeter,” “shopping_guide,” “garment”), and useful metadata (voice, sample rate, device IP). A background heartbeat thread posts to `/heartbeat` at a fixed interval with dynamic metadata (e.g., device sample rate), enabling simple health monitoring. When an agent creates a conversation, the client persists the returned *conversation_id* to `/tmp/current_conversation_id.txt`, establishing a hand-off channel between processes running at different times or on different Pis. Subsequent *log_message* and *handover* helpers standardise telemetry, so all turns and structural events are consistently captured with optional meta fields such as *interruptible_tts.py* keyword lists, utterance durations, or reasons for exit. The base URL of the Central service is fully parameterised via *CENTRAL_URL*, facilitating deploy-

ment across local networks or hosts; all functions return a simple `{ok: ...}` JSON contract and degrade gracefully on network issues.

4.3.2 Execution interface design

A thin execution interface (i.e., "main.py") process orchestrates stage transitions for Greeter/Shopping-guider role switching locally without requiring from the server. On boot, it plays a short TTS startup line (via Edge-TTS), sets system volume, and then launches the Greeter Jarvis first. When Jarvis exits after a successful handover, the execution interface starts Alice and passes along the current conversation ID via the environment. This design chooses process boundaries as the unit of stage switching, which keeps per-agent codebases independent and ensures that failures or exits in one stage do not poison long-lived state in another. The conversation ID file (`/tmp/current_conversation_id.txt`) is the only implicit coupling, and it is intentionally simple, audited via the central client, and human-inspectable during development.

4.3.3 Agents profiling

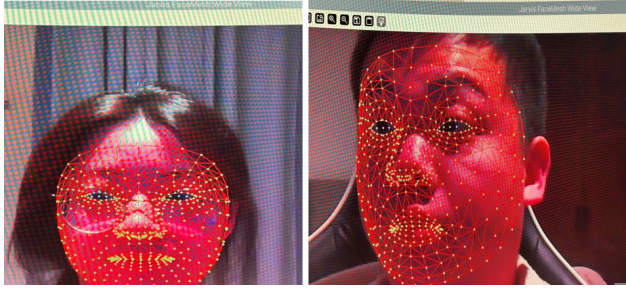


Figure 10: MediaPipe FaceMesh Human-Facial Recognition from Jarvis to identify Passerby - The two people in the picture are master’s degree students from University College London. Their facial information collection and the use of the picture have been approved

Jarvis, the mobile greeter, illustrates how the design blends perception with conversation. Using Picamera2 and MediaPipe FaceMesh (see Figure 10), Jarvis waits for a stable face-presence window (2–3 seconds) before initiating a welcome; on this threshold crossing, he begins a new conversation via the central client, speaks a short greeting that can be interrupted by a keyword, then listens for the customer’s response using VAD. A lightweight intent classifier gates the next action: if the user indicates interest (e.g., “yes”), Jarvis announces a handover to the specialist and posts a handover to the central service before exiting cleanly so the supervisor can launch Alice; if the user declines or remains silent through a nudge-and-goodbye

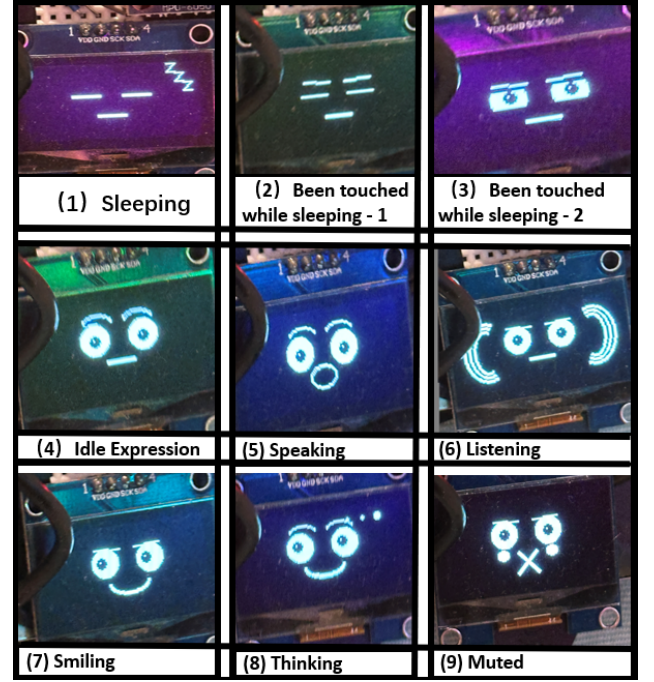


Figure 11: Garment Agent Visual Expression on OLEDs - The video for demonstrating the garment agents’ visual expression can be found on the GitHub page indicated at the start of the report

routine, Jarvis logs the terminal event and resets to idle. All assistant utterances (interruptible or not) and all user turns are logged with timestamps and meta tags, and Jarvis also registers and heartbeats on startup with capability descriptors such as [“vision”, “tts”, “stt”, “handoff”], voice name, and current IP. This makes Jarvis’s entire loop—from face detection through handover—transparent in the centralized telemetry.

Alice, the shopping-guide agent, inherits the existing conversation if one exists (so log continuity is preserved when she is launched after Jarvis), or creates a new session defensively if she starts standalone. Her dialogue manager is deliberately simple: a compact rolling chat history drives responses from a small LLM; engagement control follows the two-phase “reminder then goodbye” path; “leaving” intent is detected with a micro-prompt; and all assistant messages are produced through the interruptible TTS module with keywords enabled. Throughout, she posts heartbeat metadata (including the recording sample rate and device IP), and every assistant/user turn is mirrored to the central stream with flags that indicate interruptibility or the keyword set in effect. This yields deterministic replay in the dashboard and an auditable map from inputs to outputs during study analysis.

Nova/Bob has lightweight memory , semantic mut-

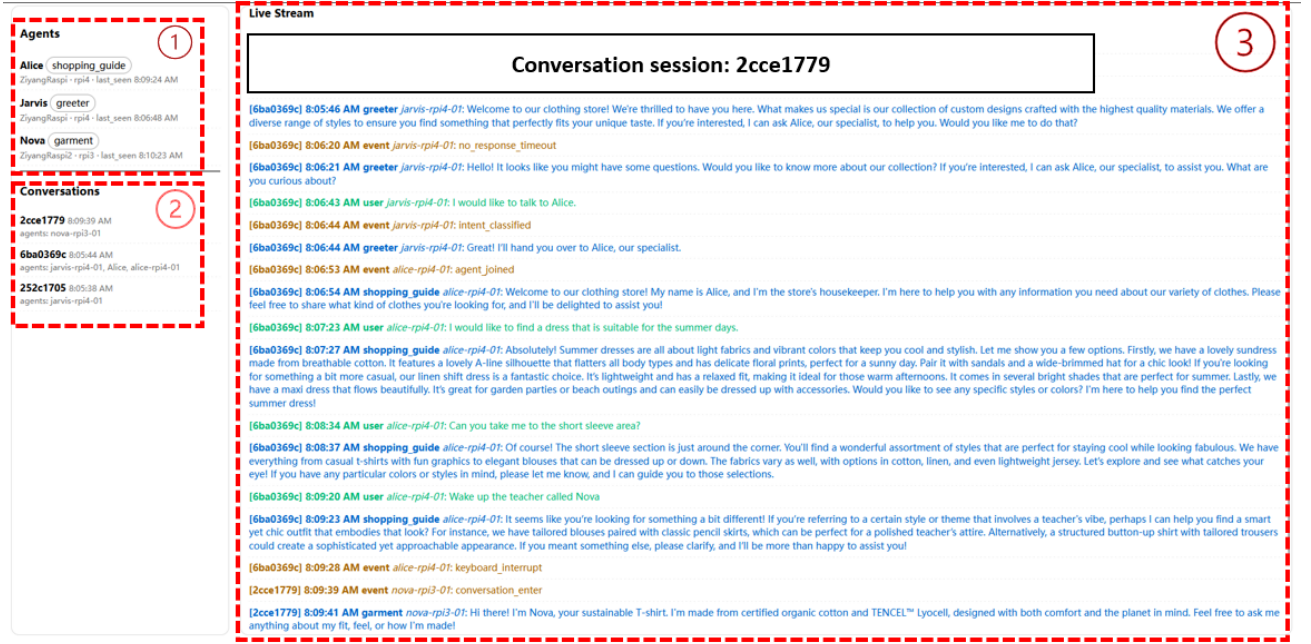


Figure 12: System Behavior Observation on "localhost:8000/dashboard" for an Example Case - There are three parts in the dashboard design: (1) Agents Card listing the current joined agents, (2) Conversations Card listing the occurred event following the operational workflow as shown in section 3.2 and (3) Live stream or chat-history for conversation session ticked on the Conversations Card.

ing and visual expression. For memory, Nova/Bob maintains a compact JSON store that aggregates stable user preferences (e.g., fit, size, materials, sensitivities) extracted from user utterances via a schema-constrained prompt; answers condition on both these preferences and fixed product facts (e.g., certified materials, care instructions) so that follow-ups become progressively tailored over the session. For muting, Nova/Bob classifies each user turn as "mute", "unmute", "none" using a small semantic prompt rather than brittle keywords; when muted, Nova/Bob enters a listen-only loop that continues to capture audio, seeking an unmute intent within a bounded window. Upon unmute, Nova/Bob drains any residual audio to prevent self-echo, thanks the user succinctly, and resumes the dialogue; a short "unmute-echo suppression" window avoids responding inadvertently to the unmute phrase itself. Nova/Bob exposes a level callback from the TTS module to drive an OLED "face" (listening/thinking/speaking/muted), and logs "muted", "unmuted", "assistant_interrupted", and "end_conversation" events with timing metadata so that the dashboard reflects not only content but also conversational dynamics. Like the other agents, Nova/Bob registers and heartbeats on launch with a capability descriptor (["tts", "stt", "sensors", "oled", "mute_semantic"]) and uses the same 16 kHz / 30 ms audio settings, VAD

mode, and keyword list for interruptible speech. The visual expression implementation details (see Figure 11) for garment agent has been explored in the hardware design section 4.2.2 for OLEDs integration.

4.3.4 Observability design

Observability is first-class in the software design part of TPG-MAS, which is needed for debugging and visualising system behaviors during development stage, one system in-operation example been observed is presented in Figure 12. The Central server's dashboard renders a live stream of the most recent activity and retro-loads recent history on connection, so researchers can watch multi-agent sessions unfold and then scroll back to analyze key decision points. The UI shows agent rosters (with types), the active conversation set, and a time-ordered message/event feed. Assistant posts can optionally display speech durations, and user posts can show utterance lengths when clients report them; handovers are formatted with from/to agent roles for clarity. Because all messages share the same shape (role, text, meta, conversation_id, server timestamp), downstream analysis scripts can be trivially built on the same HTTP list endpoints during evaluation and user-study scoring

4.3.5 Configuration and failure modes

Finally, the configuration and failure modes are intentionally simple. Environment variables control the OpenAI key and the Central base URL, and the client library prints concise diagnostics but returns a uniform `{ok: False}` on error so that agents can continue local interaction even if the Central service is temporarily unavailable. The in-memory server is appropriate for a prototype and is explicitly documented as swappable for SQLite in later iterations; the same data models and broadcast events would carry over unchanged. The design’s few file-system couplings (notably the conversation-ID file) are easy to inspect and reset during integration testing, and stage orchestration is process-local, which greatly reduces the risk of cascading failures across agents. The result is a minimal but complete software backbone for a multi-stage, multi-agent retail experience: agents remain loosely coupled, speech is truly interruptible, handovers are explicit and auditable, and the Central dashboard provides the empirical substrate for analysing user journeys at scale.

4.4 LLM-Mediated Agent Design

The LLM-mediated layer across all agents is built on a single stack: GPT-4o-mini for dialogue generation, Whisper-1 for speech transcription, and neural TTS voices for playback (Jarvis uses en-US-GuyNeural; Nova uses en-GB-SoniaNeural). The agents call the chat API with structured message arrays (system + user, optionally augmented with context blocks) and tune sampling by task: Nova answers at a conservative temperature (around 0.6, max_tokens around 180), while short one-shot prompts (greetings, nudges, goodbyes) in Jarvis/Alice are generated with higher temperatures (0.8–0.9) and tighter token budgets (around 40–120). Speech is captured at 16 kHz with WebRTC VAD endpointing and passed to Whisper-1 before each LLM turn; TTS playback is wrapped to allow barge-in. This configuration is implemented uniformly in code: Jarvis and Alice both call `openai.ChatCompletion.create(model="gpt-4o-mini")` for replies and intent checks; Nova centralizes the model and decoding defaults and reuses them across greeting, nudge, apology, answer, and farewell routines.

Prompt engineering centers on concise, role-faithful system instructions plus small, purpose-built user prompts. Alice is framed as an “elegant, warm, professional housekeeper,” instructed not to repeat herself verbatim and to vary phrasing; Jarvis is cast as a “warm store greeter” with templated welcome/reminder/goodbye prompts; Nova speaks in the first person as a sustainable T-shirt, constrained to

1–3 short sentences, no emojis, and at most one follow-up question. Task prompts are specialized: Nova switches templates by dialogue function (greeting/nudge/apology/farewell/answer), and all agents use micro-prompts for classification (e.g., YES/NO leave detection; strict-JSON mute/unmute intent) at low temperature to stabilize outputs. To avoid lexical repetition in high-exposure utterances (e.g., welcomes), the agents persist previously used lines and resample when a collision is detected. Together, these patterns keep outputs natural, brief, and interruption-friendly at the point of sale.

Context engineering operates at three layers and is shared by all agents. (i) Short-term conversational state: Alice trims and reuses the recent window of turns; Nova constructs an explicit context block that concatenates a running long-term summary with the latest exchanges to ground each response. (ii) Persistent memory: Nova extracts stable clothing preferences (size, color, fit, budget, sensitivities, sustainability priority) into a local JSON store and incrementally maintains a factual conversation summary after each exchange. (iii) Cross-agent session context: a central service manages registration/heartbeat, conversation IDs, message/event logging, and handovers so that Jarvis can initiate a session on face-presence, transfer the live conversation to Alice, and allow Nova to join from sensor triggers (pickup via IMU-6050 or long-press capacitive touch) while inheriting the same conversation context. Across agents, responses are logged with “interruptible” metadata, and LLM classifiers gate flow for user intents such as leave, mute/unmute, or decline, ensuring continuity and graceful recovery under barge-in.

5 EVALUATION & DISCUSSION

The breadth and integration complexity of the TGP-MAS, spanning embedded hardware test, software/hardware integration, multi-agent orchestration, 3D-Model designing and printing, meant that by the time the prototype reached a stable state, insufficient time remained to secure formal ethics approval for a formal user study to evaluate the system. Consequently, this project conducted an informal, small-scale pilot study to obtain indicative evidence of usability and technical performance, and to surface failure modes prior to a formal deployment.

This informal pilot study recruited five postgraduate-taught participants drawn from two programmes held by department of Computer Science at UCL: MSc Systems Engineering for the Internet of Things and MSc Artificial Intelligence and Robotics. Beyond their technical familiarity with AI-Agent and HCI, these participants were purposefully sampled for accent diversity (i.e., they originated from different first-language backgrounds) to probe the robustness of the voice interaction component to diverse English accents.

(Notes: Participants provided informed consent; only anonymised transcripts and system logs were retained. Results should be interpreted as indicative rather than statistically generalisable.)

5.1 Voice-Interaction Component Evaluation

To benchmark the voice-interaction component built on this project, three key indicators were introduced and measured: (1) Voice Recognition Accuracy, (2) Voice Recognition Time and (3) Response Time. These three indicators are measured based-on the test set contains 30 retail-oriented transcripts (10 simple, 10 medium and 10 complex sentence), which are put in Figure 15 on Appendices section A for reference. These transcripts are spoken by five accent-diverse postgraduate participants as mentioned earlier. In addition, another two ASR systems are used and replace with the Whisper deployed in the default version of voice-interaction component on TPG-MAS, they are, a lightweight local voice recogniser Vosk and Azure Speech-to-Text (STT), thus comparing their performance and analysing if there is any improvement can be made from the voice-interaction component design aspect to enhance the overall system performance in future work.

5.1.1 Voice recognition accuracy

Voice recognition accuracy is reported via Word Error Rate (WER), computed as

$$\text{WER} = \frac{S + D + I}{N},$$

where S , D , and I are word substitutions, deletions, and insertions relative to the reference transcript, and N is the number of reference words. As shown in Table 1, Whisper achieves the highest overall accuracy, while the local model is lowest (especially on complex utterances).

	Word Error Rate (%)			Test Cases
	Whisper	Vosk	Azure STT	
Simple sentence	98%	92%	97%	50
Medium sentence	95%	85%	93%	50
Complex sentence	89%	74%	87%	50
Overall (avg.)	94.0%	83.7%	92.3%	150

Table 1: Voice Recognition Accuracy Comparison among Different ASR.

5.1.2 Voice recognition time

For *voice recognition time* comparing, this project measures the time-to-final at the moment of end-of-speech detection. This is when the VAD fires, and the pipeline prints a terminal log that includes the recogniser’s elapsed processing time for that utterance. This section aggregates these terminal readings and report mean $\pm$ SD over the same 30 sentences (simple/medium/complex). As shown in Table 2, the local Vosk recogniser is consistently fastest (0.8–2.0 s), while Whisper and Azure STT exhibit similar latencies (about 3–5 s), with longer times on complex utterances.

	Recognition Time (s)			Test Cases
	Whisper	Vosk	Azure STT	
Simple sentence	3.3 $\pm$ 0.4	1.1 $\pm$ 0.3	3.3 $\pm$ 0.5	50
Medium sentence	3.9 $\pm$ 0.5	1.4 $\pm$ 0.3	4.0 $\pm$ 0.4	50
Complex sentence	4.5 $\pm$ 0.6	1.8 $\pm$ 0.4	4.7 $\pm$ 0.7	50
Overall (avg.)	3.9	1.4	4.0	150

Table 2: Average Recognition Time Comparison among Different ASR.

5.1.3 Response time

This user study reports *response time* (RT) as the post-endpointing latency from end-of-speech detection to receipt of the first stable final transcript from LLM’s response:

$$\text{RT} = t_{\text{final}} - t_{\text{eos}}.$$

The pipeline prints terminal timestamps at both events, and per-utterance RT is computed from these logs and summarised as mean $\pm$ SD over the same 30 sentences spoken by five accent-diverse postgraduate participants. As shown in Table 3, all three recognisers exhibit comparable response times within 1–2 s: Whisper $\approx$ 1.25–1.63 s across difficulties (overall $\approx$ 1.44 s), Vosk $\approx$ 1.20–1.56 s (overall $\approx$ 1.38 s), and Azure STT $\approx$ 1.27–1.65 s (overall $\approx$ 1.46 s). The tight clustering

indicates that post-endpoint processing—decoder stabilisation, punctuation/casing, and minimal network overhead—dominates similarly across systems, while larger differences appear in time-to-final end-to-end latency. Practically, this suggests UI feedback (e.g., “transcribing...” indicators) can be engineered uniformly across models, with perceived responsiveness primarily affected by pre-endpoint interaction dynamics and full decoding complexity rather than this post-endpoint RT window.

	Response Time (s)			Test Cases
	Whisper	Vosk	Azure STT	
Simple sentence	1.25 ± 0.18	1.20 ± 0.17	1.27 ± 0.19	50
Medium sentence	1.43 ± 0.22	1.38 ± 0.20	1.45 ± 0.23	50
Complex sentence	1.63 ± 0.25	1.56 ± 0.27	1.65 ± 0.26	50
Overall (avg.)	1.44	1.38	1.46	150

Table 3: Average Response Time (mean ± std.) Comparison across Different ASR.

5.2 Multi-Modal Agent Activation Evaluation

In the multi-modal activation design of the agent in TGP-MAS, there are multiple indices worth batch testing to determine the system’s proactivity to user. These indicators are: human detection accuracy and remote activation accuracy.

5.2.1 Human detection accuracy

As the entry point for user interaction with the system, the accuracy of *Jarvis*’s visual face detection is critical. The front-camera setup is: resolution 640×480 at 15FPS; detection threshold = 0.60; NMS = 0.45. A detection counts as correct if, within 500 ms after a person enters the frame, the model returns a face box with confidence ≥ 0.60 and IoU ≥ 0.50 against human annotations. Dataset: 5 participants $\times$ 12 scripted clips $\times$ 12s at 15 FPS $\rightarrow$ 10,800 person-present frames, plus 3,600 empty-scene frames for false-positive assessment. Detector: lightweight CPU face detector (YuNet/BlazeFace-class).

This user study reports Recall (i.e., “probability of detecting a person when a person is present”) as the metric. Per-condition subsets are evaluated on different clips and are not additive; the overall score is computed on the union of person-present frames.

As summarised from the data collection on Table 4: Under typical retail lighting (i.e., Bright 450-650 lux) with frontal faces at 1-2 m, Jarvis reaches 95-97% recall; performance drops in backlight, long range (about 3 m), heavy yaw, or occlusion, where recall remains 82-89%. The overall recall of 92.1% and a low empty-scene FP rate (0.12/min) are adequate for “walk-up and be greeted” multi-modal activation. Practical mitigations—camera placement to

Condition	Recall
<i>Lighting</i>	
Bright retail lighting ($\approx 450\text{--}650\text{ lux}$)	95.1%
Dim lighting ($\approx 120\text{--}200\text{ lux}$)	89.3%
Backlit / against a window	84.2%
<i>Distance</i>	
Distance $\approx 1.0\text{ m}$	96.0%
Distance $\approx 2.0\text{ m}$	92.1%
Distance $\approx 3.0\text{ m}$	82.7%
<i>Occlusion</i>	
Mask	88.5%
Cap / hood	86.1%
<i>Pose</i>	
Frontal ($\text{yaw} < 15^\circ$)	97.2%
$\sim 30^\circ$ yaw	92.4%
$\sim 60^\circ$ yaw	82.0%
<i>Motion</i>	
Stationary	94.8%
Walk-by	90.1%
Overall (weighted, 10,800 frames)	92.1%

Table 4: Jarvis Visual Face Detection Recall Under Different Conditions

avoid backlight and consensus triggers with VAD—can further reduce misses.

5.2.2 Remote activation accuracy

Alice’s ability to remotely wake-up a garment agent (Bob/Nova) during the full dialogue workflow are evaluated. Each of five participants completed three end-to-end runs (15 total). A run is counted as *successful* if, after Alice issues the remote-wake command, the target garment agent audibly greets (or displays its wake UI) within 2s; otherwise it is counted as a failure. Timestamps for the command emission and garment acknowledgment were printed to the terminal to determine outcomes.

Participant	Attempts	Successes	Failures	Accuracy
P1	3	3	0	100.0%
P2	3	3	0	100.0%
P3	3	2	1	66.7%
P4	3	3	0	100.0%
P5	3	3	0	100.0%
Total / Overall	15	14	1	93.3%

Table 5: Alice $\rightarrow$ Garment Remote Activation Results per Participant (3 trials each).

Table 5 reports per-participant results summarises the overall failure rate and accuracy. It is observed **14/15** successful remote wakes, yielding a **failure rate of 6.7%** and an overall **accuracy of 93.3%**.

The single failure occurred under transient Wi-Fi congestion; a subsequent retry (outside the 2s window) succeeded but was still counted as a failure by protocol.

5.3 User Feedback

This pilot study conducted a qualitative user evaluation to capture perceived experience with the unmanned clothing-store system. Five postgraduate participants completed the full three-stage flow and then answered a 6-construct questionnaire using 5-point Likert items (1=strongly disagree, 5=strongly agree). Constructs were: (1) Felt Understood; (2) Conversational Naturalness & Turn-Taking Comfort; (3) Role Clarity & Handover Coherence; (4) Usefulness & Recommendation Quality; (5) User Agency, Control & Recoverability; (6) Comfort, Privacy & Social Acceptability. Each construct score is the mean of two short items (e.g., “The conversation felt natural,” “I could interrupt and correct easily”), followed by one open-ended prompt

Figure 13 presents the scores for each qualitative indicators. Participants generally felt *well understood* and found *roles/handovers clear*, and they rated *comfort/privacy* positively. However, **Conversational Naturalness & Turn-Taking** was notably lower, with open-ended comments citing **slow responses** and **templated wording** that reduced flow and spontaneity. *Usefulness* and *Agency/Recoverability* were mid-range, reflecting that users could usually steer or recover but delays sometimes made corrections awkward. These findings align with the voice-interaction component latency analysis and indicate priorities for iteration: reduce end-to-end delay, add quick backchannels, diversify phrasing, and improve interruption handling.

The quotes of these participants have been anonymously posted in the Figure 16 on Appendix section A.

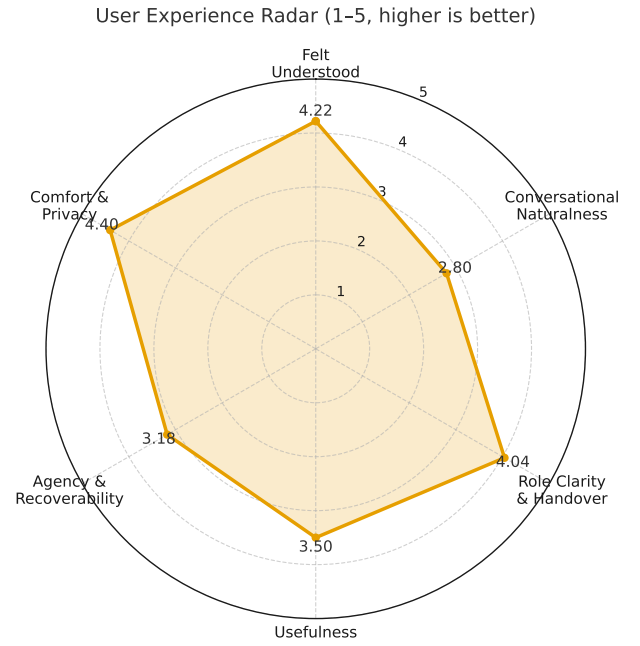


Figure 13: User Experience Evaluation Radar Plot

6 FUTURE WORK

Future work will prioritise three directions: (i) injecting principled path-planning into the mobile agents (Jarvis and Alice) so they can locate and approach different garment agents in complex store layouts; (ii) migrating the language-model stack from remote APIs to local, on-device inference; and (iii) reducing end-to-end latency in speech recognition and response to make human-agent interaction more fluid and natural.

First, for navigation, the current workaround—having Alice pause speaking for 10 seconds to trigger a garment agent—will be replaced by an explicit perception-to-plan pipeline. Concretely, we will introduce a modular navigation stack combining a global planner (e.g., grid-based A* or D*-Lite on a store costmap) with a local, reactive controller for obstacle avoidance, backed by a semantic floor plan that encodes aisles, fixtures, and no-go regions. The Central Agent will expose garment-agent locations as dynamic targets, updated via registration beacons or visual/RFID/AprilTag cues, allowing Jarvis/Alice to sequence multi-target search and handover. Human-aware navigation (speed/clearance modulation near shoppers), online map updates for temporary obstructions, and failure-recovery behaviours (re-plan, alternative routes, or remote assistance) will be integrated. We will evaluate navigation by time-to-reach, path optimality, success rate across varied layouts, and the impact on dialogue continuity during movement.

Second, it is pursued a local LLM deployment on agents to improve privacy, cost predictability, and resilience to network variability. The plan is to adopt a lightweight, quantised model (e.g., 4–8-bit, 7B-class) with domain-specific retrieval-augmented generation from an on-device garment knowledge store and store policies. Distillation from the current API model will preserve task performance while fitting edge constraints. A tiered architecture will allow graceful degradation—defaulting to the local model and optionally escalating to a larger server model only when confidence falls below a threshold. Prompt scaffolds, tool-use policies, and safety filters will be co-designed for the local stack to keep behaviours consistent with the current system.

Third, to reduce voice-interaction latency, we will re-architect the audio loop for streaming and concurrency. On the ASR side, we will use frame-level VAD and endpointing with partial hypotheses emission, low-footprint/quantised models, and optional hardware acceleration. On the NLU/LLM side, we will cache system prompts and retrieved passages, pre-compile tool schemas, and pipeline decoding so TTS can begin once a high-confidence prefix is available

(“speak-as-you-think” with safe barge-in). For TTS, we will adopt low-latency vocoders and cache frequent utterance templates (greetings, confirmations). Transport will move from sequential REST calls to persistent streams (e.g., WebSocket-style RPC) to avoid connection setup overhead. We will benchmark latency as user-to-speech round-trip time and measure perceived naturalness in a controlled user study.

Beyond these priorities, several extensions are planned: (a) a digital-twin simulator of store layouts to stress-test path planners and dialogue policies before deployment; (b) multi-modal perception (vision + tags + BLE/RFID) for robust garment-agent discovery; (c) adaptive multi-agent scheduling that jointly optimises path length, queueing, and conversational context retention; (d) multilingual and accessibility-first interaction (noise-robust ASR, clear-speech TTS, haptics/visual captions on displays); (e) persistent, privacy-preserving user profiles to personalise recommendations while complying with data-minimisation principles; and (f) energy/performance instrumentation to quantify the carbon and compute footprint of on-device inference. Together, these directions aim to make TGP-MAS more autonomous in space, more reliable offline, and significantly more natural in real-time human-agent collaboration.

7 CONCLUSION

This project delivers a working proof-of-concept for an unmanned clothing-store environment built around an LLM-mediated, multi-agent architecture. It realises an end-to-end pipeline—from greeting and guidance to garment-specific interaction—through a set of physically embodied agents with distinct roles, a modular software stack, lightweight prompt and context-understanding mechanisms, and a centralised service for registration, health monitoring, conversation logging, and agent handover. Together, these components form a coherent baseline capable of orchestrating human-agent dialogues in a retail setting and demonstrate the feasibility of “talking garments” as an interaction paradigm.

Functionally, the system shows that separating concerns across agents (store greeter/guide vs. garment-embedded agents) and coordinating them through a central orchestrator is an effective pattern for real-world deployment. The design emphasises pluggability at both the hardware and software layers: sensors, ASR/TTS back-ends, and language models can be swapped without rewriting core logic; agents can be added or removed while remaining discoverable and addressable by the central service. Early trials in a mock store layout confirm that multi-agent handovers, state awareness, and basic context reten-

tion are achievable with minimal engineering overhead when supported by disciplined prompt scaffolds and a shared conversational memory.

At the same time, the current prototype surfaces clear limitations that frame the next stage of work. Conversational naturalness is uneven: agents occasionally produce off-target or overly literal responses, and the system is sensitive to phrasing outside the prompted patterns. End-to-end latency in speech interaction remains higher than desirable, which affects turn-taking fluidity and perceived responsiveness. Navigation and spatial coordination are functional but simplified, relying on heuristic triggers rather than a principled path-planning stack grounded in a semantic map of the store. Finally, the evaluation to date has been small-scale and formative; broader testing with diverse users and environmental conditions is needed to validate robustness and generalisability.

Despite these constraints, several contributions are established. The project (i) articulates a modular blueprint for LLM-mediated retail agents that other sites can replicate; (ii) demonstrates garment-embedded interfaces as a viable complement to mobile service agents; (iii) integrates a central registry/heartbeat and conversation logger that make multi-agent behaviour observable and debuggable; and (iv) provides a minimal, transparent approach to prompt and context engineering that can be systematically extended. Importantly, the system underscores that pragmatic engineering choices—clear role definitions, a shared state bus, and conservative handover policies—often yield more reliability than increasing model complexity alone.

Looking ahead, the most impactful improvements will come from three directions outlined in the future work: principled path planning and human-aware navigation for Jarvis and Alice to operate in complex layouts; a migration toward local or on-prem LLM inference for privacy, predictability, and resilience; and a streaming, concurrency-first audio loop to reduce recognition and response latency. Beyond these, we foresee gains from a digital-twin simulator for rapid iteration on navigation and dialogue policies; multi-modal garment discovery (vision, RFID/BLE, tags) to reduce reliance on speech alone; stronger guardrails and retrieval-augmented grounding to mitigate off-topic outputs; and expanded accessibility features (noise-robust ASR, captioning, haptic cues). With these advances, the system can progress from a robust prototype to a deployable platform for natural, sustainable, and scalable unmanned retail experiences.

Acknowledgements

First of all, I would like to express my gratitude to Professor Marianna. It was a great fortune for me to have her as my personal tutor, group project supervisor and independent research project supervisor throughout this year of my master’s study. I am very grateful for her idea-sharing, academic insights, patient guidance and continuous follow-up, which enabled me to successfully complete two of the most difficult tasks in this year. Her help to me is not only in academic terms, but also made me reflect on my sleeping habits and self-management skills, giving me more confidence to face the challenges of my future career and life.

I would also like to express my gratitude to my co-supervisor, Ms. Shu. She provided me with invaluable insights for my individual research project. Her approachable manner and supervisory style made my experience on this research project much more enjoyable and also motivated me to complete it.

Finally, I would like to express my gratitude to all the faculty members of the MSc Systems Engineering for the Internet of Things program, including Dr. Akin, Dr. Elia, Dr. Valerio, and Mr. Bonot. They have provided me with a solid technical foundation, which is not only applicable to my research project but also beneficial for my future career.

Thank you all for giving me an unforgettable Master’s journey!

References

- [1] Kushal Agrawal et al. “Evaluating LLM Agent Collusion in Double Auctions”. In: *arXiv preprint arXiv:2507.01413* (2025). Version v1 published 2 July 2025. arXiv: 2507.01413 [cs.GT]. URL: <https://arxiv.org/abs/2507.01413>.
- [2] Elijah Atamas. *AI Voice Agents: Real-Time vs Turn-Based (TTS/STT) Architecture*. Softcery.com Knowledge. Compares traditional turn-based voice agents with emerging real-time streaming architectures. May 2025. URL: <https://softcery.com/lab/ai-voice-agents-real-time-vs-turn-based-tts-stt-architecture/>.
- [3] Tom B. Brown et al. “Language Models are Few-Shot Learners”. In: *arXiv preprint arXiv:2005.14165* (2020). Version v4 revised 22 July 2020. DOI: 10.48550/arXiv.2005.14165. arXiv: 2005.14165 [cs.CL]. URL: <https://arxiv.org/abs/2005.14165>.

- [4] Daniel D. Corkill. “Blackboard Systems”. In: *AI Expert* 6.9 (Sept. 1991), pp. 40–47. URL: <https://www.cs.uni.edu/~wallingf/teaching/162/readings/blackboard-systems.pdf>.
- [5] Dify Documentation. *Introduction*. Dify Docs. Introduction page of the official Dify documentation. 2025. URL: <https://docs.dify.ai/en/introduction>.
- [6] European Parliament. *The impact of textile production and waste on the environment (infographic)*. European Parliament website. Infographic summarizing environmental impacts of textile production and waste: water pollution, microplastics, carbon emissions, low recycling rates. Dec. 2020. URL: <https://www.europarl.europa.eu/news/en/headlines/society/202108ST093327/the-impact-of-textile-production-and-waste-on-the-environment-infographic>.
- [7] Juraj Gottweis and Vivek Natarajan. *Accelerating Scientific Breakthroughs with an AI Co-Scientist*. Google Research Blog. A multi-agent AI system built on Gemini 2.0 to assist scientists in generating hypotheses and research proposals. Feb. 2025. URL: <https://research.google/blog/accelerating-scientific-breakthroughs-with-an-ai-co-scientist/>.
- [8] Taicheng Guo et al. “Large Language Model based Multi-Agents: A Survey of Progress and Challenges”. In: *arXiv preprint arXiv:2402.01680* (2024). URL: <https://arxiv.org/abs/2402.01680>.
- [9] Shanshan Han et al. “LLM Multi-Agent Systems: Challenges and Open Problems”. In: *arXiv preprint arXiv:2402.03578* (2025). arXiv: 2402.03578 [cs.MA]. URL: <https://arxiv.org/abs/2402.03578>.
- [10] Hiromitsu Hattori, Arata Kato, and Mamoru Yoshizoe. “Integrating Large Language Models into Agent Models for Multi-Agent Simulations: Preliminary Report”. In: *Proceedings of the 2024 Winter Simulation Conference (WSC)*. 2024, pp. 230–241. DOI: 10.1109/WSC63780.2024.10838923. URL: <https://doi.org/10.1109/WSC63780.2024.10838923>.
- [11] Yiteng Arden Huang, Turaj Z. Shabestary, and Alexander Gruenstein. “Hotword Cleaner: Dual-microphone Adaptive Noise Cancellation with Deferred Filter Coefficients for Robust Keyword Spotting”. In: *Proceedings of the 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2019, pp. 6346–6350. DOI: 10.1109/ICASSP.2019.8682682. URL: <https://doi.org/10.1109/ICASSP.2019.8682682>.
- [12] Hiroaki Kitano et al. “RoboCup: A Challenge Problem for AI”. In: *AI Magazine* 18.1 (Mar. 1997), p. 73. DOI: 10.1609/aimag.v18i1.1276. URL: <https://doi.org/10.1609/aimag.v18i1.1276>.
- [13] Takeshi Kojima et al. “Large Language Models are Zero-Shot Reasoners”. In: *arXiv preprint arXiv:2205.11916* (2022). arXiv: 2205.11916. URL: <https://arxiv.org/abs/2205.11916>.
- [14] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *arXiv preprint arXiv:2005.11401* (2020). arXiv: 2005.11401. URL: <https://arxiv.org/abs/2005.11401>.
- [15] Guohao Li et al. “CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society”. In: *arXiv preprint arXiv:2303.17760* (2023). Accepted at NeurIPS 2023, version v2 released November 2023. DOI: 10.48550/arXiv.2303.17760. arXiv: 2303.17760 [cs.AI]. URL: <https://arxiv.org/abs/2303.17760>.
- [16] Jonathan Light et al. “AvalonBench: Evaluating LLMs Playing the Game of Avalon”. In: *arXiv preprint arXiv:2310.05036* (2023). Submitted 8 Oct 2023, revised 8 Nov 2023. arXiv: 2310.05036 [cs.AI]. URL: <https://arxiv.org/abs/2310.05036>.
- [17] Pengfei Liu et al. “Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing”. In: *ACM Computing Surveys* 55.9 (2023), pp. 1–35. DOI: 10.1145/3560815. URL: <https://doi.org/10.1145/3560815>.
- [18] Qijiong Liu et al. “Multimodal Pretraining, Adaptation, and Generation for Recommendation: A Survey”. In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD ’24)*. 2024, pp. 6566–6576. DOI: 10.1145/3637528.3671473. URL: <https://doi.org/10.1145/3637528.3671473>.
- [19] Lingrui Mei et al. “A Survey of Context Engineering for Large Language Models”. In: *arXiv preprint arXiv:2507.13334* (2025). Version v2 submitted July 21, 2025. DOI: 10.48550/arXiv.2507.13334. arXiv: 2507.13334 [cs.CL]. URL: <https://arxiv.org/abs/2507.13334>.

- [20] Joon Sung Park et al. “Generative Agents: Interactive Simulacra of Human Behavior”. In: *arXiv preprint arXiv:2304.03442* (2023). arXiv: 2304.03442. URL: <https://arxiv.org/abs/2304.03442>.
- [21] Jinghua Piao et al. “AgentSociety: Large-Scale Simulation of LLM-Driven Generative Agents Advances Understanding of Human Behaviors and Society”. In: *arXiv preprint arXiv:2502.08691* (2025). Version v1 published February 2025. arXiv: 2502.08691 [cs.SI]. URL: <https://arxiv.org/abs/2502.08691>.
- [22] Chen Qian et al. “ChatDev: Communicative Agents for Software Development”. In: *arXiv preprint arXiv:2307.07924* (2024). arXiv: 2307.07924 [cs.SE]. URL: <https://arxiv.org/abs/2307.07924>.
- [23] Sujatha R. “What are AI-Powered Voice Assistants? Beyond Basic Commands”. In: *DigitalOcean Blog* (May 2025). Discusses ASR, NLP, reasoning-capable voice agents, and voice commerce trends. URL: <https://www.digitalocean.com/resources/articles/ai-powered-voice-assistant>.
- [24] Retell AI Glossary. *Multi-Turn Conversation*. Retell AI Glossary. Defines multi-turn dialogue as context-retentive, goal-oriented conversational exchanges essential for natural AI voice agents. 2025. URL: <https://www.retellai.com/glossary/multi-turn-conversation/>.
- [25] Timo Schick et al. “Toolformer: Language Models Can Teach Themselves to Use Tools”. In: *arXiv preprint arXiv:2302.04761* (2023). arXiv: 2302.04761. URL: <https://arxiv.org/abs/2302.04761>.
- [26] Katie Seaborn, Norihisa P. Miyake, and Mihoko Otake-Matsuura. “Voice in Human-Agent Interaction: A Survey”. In: *ACM Computing Surveys* 54.4 (2021), Article No. 81. DOI: 10.1145/3386867. URL: <https://doi.org/10.1145/3386867>.
- [27] W. Shen et al. “Applications of Agent-Based Systems in Intelligent Manufacturing: An Updated Review”. In: *Advanced Engineering Informatics* 20.4 (Oct. 2006), pp. 415–431. DOI: 10.1016/j.aei.2006.05.004. URL: <https://www.sciencedirect.com/science/article/pii/S1474034606000292>.
- [28] Yoav Shoham and Kevin Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, Feb. 2009. ISBN: 9780521899437. DOI: 10.1017/CB09780521899437. URL: <https://www.cambridge.org/9780521899437>.
- [29] R. G. Smith. “The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver”. In: *IEEE Transactions on Computers* C-29.12 (Dec. 1980), pp. 1104–1113. DOI: 10.1109/TC.1980.1675516. URL: <https://doi.org/10.1109/TC.1980.1675516>.
- [30] Theodore R. Sumers et al. “Cognitive Architectures for Language Agents”. In: *arXiv preprint arXiv:2309.02427* (2023). Version v3 (TMLR camera-ready) released March 15, 2024. DOI: 10.48550/arXiv.2309.02427. arXiv: 2309.02427 [cs.AI]. URL: <https://arxiv.org/abs/2309.02427>.
- [31] VoiceSpin Glossary. *What is Voice Activity Detection (VAD) in AI Voice Bots/AI Voice Agents?* VoiceSpin Glossary. Defines VAD and explains its importance for natural, efficient voice interactions. Aug. 2025. URL: <https://www.voicespin.com/glossary/voice-activity-detection/>.
- [32] Liang Wang, Wei Zhao, and Jingming Liu. “Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models”. In: *arXiv preprint arXiv:2305.04091* (2023). arXiv: 2305.04091. URL: <https://arxiv.org/abs/2305.04091>.
- [33] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *arXiv preprint arXiv:2201.11903* (2022). arXiv: 2201.11903. URL: <https://arxiv.org/abs/2201.11903>.
- [34] Qingyun Wu et al. *AutoGen: Enabling NextGen LLM Applications via Multi-Agent Conversation*. 2023. DOI: 10.48550/arXiv.2308.08155. arXiv: 2308.08155 [cs.AI]. URL: <https://arxiv.org/pdf/2308.08155>.
- [35] Shu Zhong et al. “LLM-Mediated Domain-Specific Voice Agents: The Case of TextileBot”. In: *Behaviour & Information Technology* 44.1 (2025), pp. 1–33. DOI: 10.1080/0144929X.2025.2456667. URL: <https://doi.org/10.1080/0144929X.2025.2456667>.
- [36] Denny Zhou et al. “Least-to-Most Prompting Enables Complex Reasoning in Large Language Models”. In: *arXiv preprint arXiv:2205.10625* (2022). arXiv: 2205.10625. URL: <https://arxiv.org/abs/2205.10625>.

A APPENDICES

Figure 14: TGP-MAS Operation Workflow Illustration Version

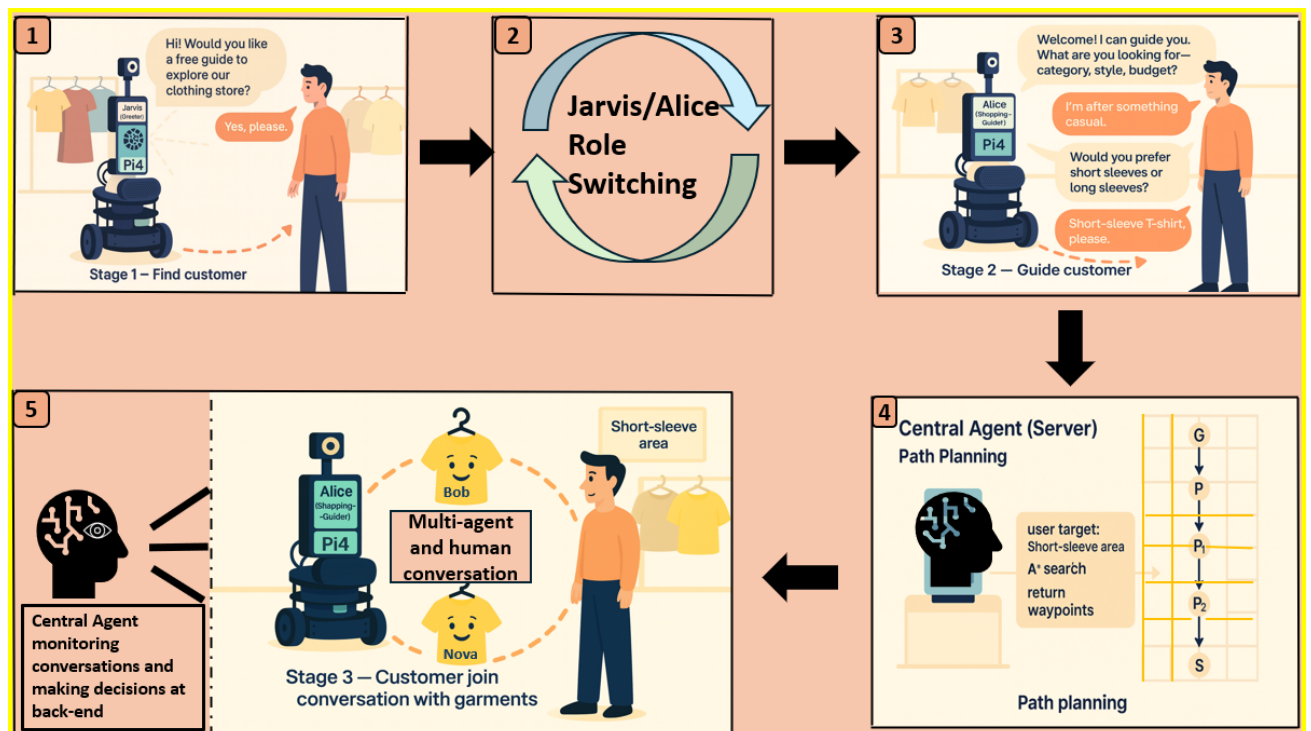


Figure 15: The Test Set of 30 retail-oriented Transcripts for Evaluating the Voice-Interaction Component of TGP-MAS

A. (10) Simple sentence

1. Hi, I'm looking for T-shirts.
2. Where is the fitting room?
3. Show me medium size.
4. Do you take card?
5. How much is this dress?
6. I like blue.
7. I need a receipt.
8. What are today's deals?
9. Can I try this on?
10. Thanks, that's all.

B. (10) Medium sentence

1. Do you have this jacket in navy, size small?
2. Please check if there's stock in the back.
3. What's your return policy for discounted items?
4. Which fabrics breathe better in hot weather?
5. I prefer organic cotton over polyester blends.
6. Recommend a shirt under fifty pounds for office wear.
7. Does this sweater shrink after machine washing at thirty degrees?
8. Can I pay with Apple Pay or contactless?
9. Which color matches beige trousers and white sneakers?
10. I'm buying a gift; can you wrap it for me?

C. (10) Complex sentence

1. I'm looking for a lightweight, water-repellent shell that layers over a chunky jumper without feeling tight, ideally under one hundred and twenty pounds.
2. If the new drop of recycled-cotton tees arrives before Friday, text me, because I'll be back to compare cuts and necklines.
3. For running, I need shorts with a zip pocket large enough for a Pixel 8 Pro and a soft waistband that doesn't chafe.
4. Could you measure the chest on the medium and the large—my chest is ninety-six centimeters—and tell me which fits closer?
5. Between merino and bamboo viscose, which keeps odor down after two days of travel without washing?
6. I'd like a blazer that works with both smart-casual Fridays and formal interviews, preferably in dark navy with subtle texture.
7. If this coat is dry-clean only, I'll pass; I need something machine-washable at thirty degrees and quick to dry.
8. When I scan the QR tag, the sustainability report should list dye process, water use, and supplier certifications, right?
9. Please compare the slim and regular fits, including sleeve length and shoulder width, because my arms are relatively long.
10. I'm deciding between size small and medium; if I raise my arms, the hem shouldn't ride up more than two centimeters.

Figure 16: User Experience Quotes

Here are the five participant quotes for the experience on the system:

P1: "It understood what I was asking—even with my accent. The replies took a little too long, so the chat felt a bit stiff."

P2: "I always knew which agent was talking and why; handovers made sense. The responses sounded scripted and arrived a beat late."

P3: "Recommendations were okay for price and materials, but I had to wait, and fixing a misunderstanding took a few extra turns."

P4: "I felt comfortable using it in the store and not worried about privacy. If it answered quicker and with more variety, I'd use it a lot."

P5: "Interrupting to correct size worked, but the flow didn't recover smoothly afterward. Still, it generally understood me well."